Linear Mixed Models (LMMs) Notes

Daniel J. Eck

Contents

Introduction 1

Basic Linear Random Effect Model Example 2

More General LMM Setup 3
- Example: Brightness of Paper Pulp Depending on Shift Operator . . . 4
- Inferential Challenges for Nested Models . . . 5
- Example: Brightness of Paper Pulp Depending on Shift Operator (continued) . . . 6
- Predicting Random Effects . . . 7

Split Plot Designs 11
- Example: Irrigation Split Plot Design . . . 11

Example: Modeling Photoprotection Dynamics in Soybean 15

Example: Nested Random Effects in a Laboratory Setting 25

Repeated Measures and Longitudinal Data 27
- Example: Panel Study of Income Dynamics . . . 28

How `lme4` fits Mixed-Effects Models 32
- Algebraic and Computational Accounting of Mixed-Model Input Formulas . . . 33
- Constructing the Random-Effects Model Matrix . . . 34
- Constructing the Relative Covariance Factor . . . 35
- Objective Function . . . 35
- Penalized Least Squares (PLS) . . . 36
- Probability Densities . . . 37
- Evaluating and Profiling the Deviance . . . 37
- The REML Criterion . . . 38

Acknowledgments 38

Introduction

So far in this class we have discussed models that include fixed effects only. In such models, the regression parameters are interpreted as population-level or sub-population-level quantities, and our inference is directed toward these fixed parameters.

However, many common statistical models can be expressed as linear models that incorporate both *fixed effects*, which are parameters associated with an entire population or with certain repeatable levels of experimental factors, and *random effects*, which are associated with individual units drawn at random from a population. Random effects are important in contexts involving subject-to-subject heterogeneity that, while a realistic

feature of the data-generating process, is not itself the primary focus of the analysis. For example, results from a large-scale agricultural experiment investigating the effect of fertilizer choice on crop yield may depend on the particular fields selected for the experiment. However, these fields are not static. Implementing the results of this experiment in practice would involve different fields entirely. Thus, fields are a structural nuisance whose individual effects are not themselves of interest. It is therefore sensible to assume that field effects arise from a common probability distribution rather than treating each as a separate fixed parameter.

A model with both fixed effects and random effects is called a *mixed-effects model*. The random effects in a mixed-effects model can either be *nested* or *crossed*. When the levels of one factor vary only within the levels of another factor, that factor is said to be nested. For example, when measuring the performance of workers at several different job locations, if the workers only work at one location, the workers are nested within the locations. If the workers work at more than one location, then the workers are crossed with locations.

We now introduce a simple example of a mixed-effects model, which includes both fixed and random effects. Our illustrative example is a two-way ANOVA model with the following structure:

$$y_{ijk} = \mu + \tau_i + v_j + \varepsilon_{ijk}$$

where μ is a population mean fixed effect, τ_i is a sub-population fixed-effect, v_j is a random effect, and ε_{ijk} is an error term. We will assume that $v_j \overset{iid}{\sim} N(0, \sigma_v^2)$ and $\varepsilon_{ijk} \overset{iid}{\sim} N(0, \sigma^2)$ where v_j and ε_{ijk} are independent. In this model, we are typically interested in:

- Estimating and testing the fixed effects τ_i, for example, testing $H_0 : \tau_i = 0$.
- Estimating the variance component σ_v^2 and testing $H_0 : \sigma_v^2 = 0$.

Notice that for fixed effects (like the τ_i), we estimate each parameter and test them individually. For the random effects, we are usually interested in estimating and testing the variance component σ_v^2 as a whole, rather than the individual v_j terms.

Basic Linear Random Effect Model Example

Let's now consider the simplest possible random effects model: a one-way ANOVA with random group effects. This is a special case of the model above, where there is only one factor with a levels:

$$y_{ij} = \mu + \alpha_i + \varepsilon_{ij}; \quad i = 1, \ldots, a; \quad j = 1, \ldots, n,$$

where the α's and the ε's have zero mean and respective variances σ_α^2 and σ^2. These variances are known as the **variance components**. Note that this formulation induces an intraclass correlation between observations at the same level that is formally expressed as

$$\rho = \frac{\sigma_\alpha^2}{\sigma_\alpha^2 + \sigma^2}.$$

This intraclass correlation quantifies the proportion of total variance that is attributable to differences between groups, and it is an important feature of random effects models.

To analyze this model using ANOVA techniques, and for simplicity, assume that each group has an equal number of observations, n per level. The total variation in the data can be decomposed as sum-of-squares total (SST) = sum-of-squres within groups (SSE) + sum-of-squares between groups (SSA),

$$\sum_{i=1}^{a}\sum_{j=1}^{n}(y_{ij} - \bar{y}_{++})^2 = \sum_{i=1}^{a}\sum_{j=1}^{n}(y_{ij} - \bar{y}_{i+})^2 + \sum_{i=1}^{a}\sum_{j=1}^{n}(\bar{y}_{i+} - \bar{y}_{++})^2,$$

where $\bar{y}_{i+}$ is the sample mean within group i and $\bar{y}_{++}$ is the overall mean across all observations,

Dividing through by the respective degrees of freedom, we obtain the mean squares, MSE and MSA. Now we find that

$$E(SSE) = a(n-1)\sigma^2, \qquad E(SSA) = (a-1)(n\sigma_\alpha^2 + \sigma^2)$$

which use the estimators:

$$\hat{\sigma}^2 = \frac{SSE}{a(n-1)} = MSE, \qquad \hat{\sigma}^2_\alpha = \frac{SSA/(a-1) - \hat{\sigma}^2}{n} = \frac{MSA - MSE}{n}.$$

These estimators are known as the ANOVA estimators, and this method of estimating the variance components can be used for more complicated designs. These estimators have the advantage that they take explicit form (a major advantage in the precomputing days), but they have a number of disadvantages including:

1. the possibility of negative variance estimates
2. not easily adapted to unbalanced data
3. the need for complex algebraic complications in more complex models

One technique for estimating parameters in mixed-effect regression models that alleviates these concerns is the familiar method of maximum likelihood estimation. Of course, this requires that we assume distributions for the random effects and the errors.

More General LMM Setup

We now turn to the general framework for linear mixed models under normality assumptions. The model is given by:

$$Y = X\beta + Zb + \varepsilon \quad \text{or} \quad Y|b \sim N(X\beta + Zb, \sigma^2 I),$$

where $Y \in \mathbb{R}^n$ is the response vector, $X \in \mathbb{R}^{n\times p}$ is a fixed model matrix, $\beta \in \mathbb{R}^p$ is a coefficient vector and is considered a fixed effect, $Z \in \mathbb{R}^{n\times q}$ is a fixed model matrix for the random effects, $b \in \mathbb{R}^q$ is a vector of random effects, and $\varepsilon \sim N(0, \sigma^2 I)$. If we further assume that the random effects satisfy $b \sim N(0, \sigma^2 D)$ and b and ε are independent, then the unconditional response Y has the distribution

$$Y \sim N(X\beta, \sigma^2(I + ZDZ^T)).$$

If D were known, we could estimate β directly using generalized least squares. In practice, however, D is unknown and must be estimated from the data. To do so, let $V = I + ZDZ^T$, so that the log-likelihood for this model is

$$l(\beta, \sigma, D|y) \propto \frac{\log|\sigma^2 V|}{2} + \frac{(Y - X\beta)^T V^{-1}(Y - X\beta)}{2\sigma^2}.$$

In principle the above can be estimated by maximum likelihood estimation, but it is difficult in practice. One difficulty is that the final unrestricted estimate may estimate some of the variances in D to be negative, and such estimates must be constrained to be zero. First order optimization conditions may therefore not be satisfied.

MLE also suffers bias when estimating random effect parameters. Recall a simple example where $x_1, \ldots, x_n \overset{iid}{\sim} N(\mu, \sigma^2)$, then the MLE of σ^2 is $\hat{\sigma}^2 = n^{-1}\sum_i (x_i - \bar{x})^2$. This estimate is biased, an unbiased estimator is leading term should be $(n-1)^{-1}$ instead of n^{-1}. Conceptually, these problems are driven by estimation of μ. If μ were known then $n^{-1}\sum_i (x_i - \mu)^2$ is an unbiased estimator for σ^2. Similar problems occur with the estimation of variance components. Given that the number of observations for some factor may not be large, the bias of the MLE for the variance component associated with that factor may be quite large.

Restricted maximum likelihood estimation (REML) estimators are an attempt to get around problems with standard unrestricted maximum likelihood estimation. The idea of REML is to find all linear combinations, denoted k, such that $k^T X = 0$. Form a matrix K with columns k, so that

$$K^T Y \sim N(0, K^T V K).$$

This removes the effect of estimating the mean (fixed effects) from the problem. We can then proceed to maximize the likelihood based on $K^T Y$. Once the random effect parameters have been estimated, it is easy enough to estimate the fixed effect parameters via plug-in. Plug-in gives,

$$Y \approx N(X\beta, \sigma^2 \widehat{V}_{\text{REML}}),$$

and maximum likelihood then gives

$$\hat{\beta}_{\text{REML}} = \left(X^T \widehat{V}_{\text{REML}}^{-1} X\right)^{-1} X^T \widehat{V}_{\text{REML}}^{-1} Y.$$

REML generally produces less biased estimates than unconditional MLE. For balanced data, REML estimates are usually the same as ANOVA estimates.

Example: Brightness of Paper Pulp Depending on Shift Operator

We illustrate the fitting methods using data from an experiment testing how paper brightness is influenced by the shift operator on duty. See Section 8.1 in Faraway [2016] for more details. We begin with a fixed effects one-way ANOVA.

```
library(ggplot2)
library(faraway)
data(pulp)
op = options(contrasts = c("contr.sum", "contr.poly"))
```

By default, R uses treatment coding (`contr.treatment`), where the intercept represents the mean of a reference group and all other levels are compared against it. Setting `contr.sum` switches to sum-to-zero coding, where the intercept represents the grand mean and each factor coefficient represents a group's deviation from that grand mean. This aligns more naturally with the ANOVA decomposition and avoids the arbitrariness of a reference level, which is why it is preferred when fitting mixed-effects models. The result is stored in `op` so the original contrast settings can be restored later with `options(op)`.

```
lmod = aov(bright ~ operator, pulp)
summary(lmod)
```

```
##             Df Sum Sq Mean Sq F value Pr(>F)
## operator     3   1.34  0.4467   4.204 0.0226 *
## Residuals   16   1.70  0.1062
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
coef(lmod)
```

```
## (Intercept)   operator1   operator2   operator3
##       60.40       -0.16       -0.34        0.22
```

We see that the p-value for the operator effect is roughly 0.023. We have $\hat{\sigma}^2 = 0.106$ and the overall mean is 60.40. Under sum-to-zero coding, $\sum_i \alpha_i = 0$, so the effect for the fourth operator can be recovered as $-(0.16 + 0.34 - 0.22) = -0.28$. Turning to the random effects model, we can estimate the variance of the operator effects $\hat{\sigma}_\alpha^2$ using the formula above as

```
## (MSA - MSE) / n
(0.4467 - 0.1062)/5
```

```
## [1] 0.0681
```

We now demonstrate the MLE approach using REML. We will use the `lme4` package to accomplish this task.

```
library(lme4)
```

```
## Loading required package: Matrix
```

```
mmod = lmer(bright ~ 1 + (1|operator), pulp)
summary(mmod)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: bright ~ 1 + (1 | operator)
```

```
##    Data: pulp
##
## REML criterion at convergence: 18.6
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -1.4666 -0.7595 -0.1244  0.6281  1.6012
##
## Random effects:
##  Groups   Name        Variance Std.Dev.
##  operator (Intercept) 0.06808  0.2609
##  Residual             0.10625  0.3260
## Number of obs: 20, groups:  operator, 4
##
## Fixed effects:
##             Estimate Std. Error t value
## (Intercept)  60.4000     0.1494   404.2
```

In the above, the model fitted has both fixed and random effects components. The fixed effect here is just the intercept represented by the first `1` in the model formula. The random effect is represented by the `(1|operator)` indicating that the data is grouped by the `operator` variable, and the `1` indicates that the random effect is constant within each group.

The default fitting method is the REML procedure mentioned above. We see that this method gives identical estimates to the ANOVA results above. For unbalanced designs, the REML and ANOVA estimators are not necessarily identical.

Inferential Challenges for Nested Models

Using standard likelihood theory, we may derive a test to compare two nested hypotheses, H_0 and H_1, by computing the likelihood ratio test statistic:

$$2\{l(\hat{\beta}_1, \hat{\sigma}_1^2, \hat{D}_1|Y) - l(\hat{\beta}_0, \hat{\sigma}_0^2, \hat{D}_0|Y)\}$$

where $\hat{\beta}_0, \hat{\sigma}_0^2, \hat{D}_0$ are the MLEs of the parameters under the null hypothesis and $\hat{\beta}_1, \hat{\sigma}_1^2, \hat{D}_1$ are the MLEs of the parameters under the alternative hypothesis. The distribution of the above test statistic

$$2\{l(\hat{\beta}_1, \hat{\sigma}_1^2, \hat{D}_1|Y) - l(\hat{\beta}_0, \hat{\sigma}_0^2, \hat{D}_0|Y)\} \approx \chi^2_{\text{df}}$$

where the degrees of freedom is equal to difference in the dimensions of the two parameter spaces (the difference in the number of parameters when both models are identifiable). Note that testing in this manner is approximate and that the parameters under the null hypothesis are on the boundary of the parameter space. This is a real concern when testing the hypothesis about the random effects that take the form $H_0 : \hat{\sigma}^2 = 0$. Furthermore, the χ^2 approximation can be poor even when the conditions are met. The p-values generated by the LRT for fixed effects are approximate and unfortunately tend to be too small, thereby sometimes overstating the importance of some effects.

Note that we may use bootstrap methods to find more accurate p-values for the LRT. The most popular bootstrap approach is nonparametric in that no distributional assumptions are made. Since we are willing to assume normality for the errors and the random effects in our modeling formulation, we can use a technique called the **parametric bootstrap**. In this technique, we generate data under the null model using the fitted parameter estimates as guesses for the true parameter values. We compute the LRT test statistic for this generated data. We repeat this scheme many times and compare the observed LRT test statistic to the distribution of LRT test statistics obtained from repeatedly generation of samples. We will discuss this approach below. Here is a useful concise reference on bootstrap procedures.

An alternative approach is to condition on the estimated values of the random effect parameters and then use standard F- or t-tests. This assumes that the covariance of the random part of the model D is equal to its estimated value.

- **Testing random effects**: In most cases testing will be of the form $H_0 : \sigma^2 = 0$. This breaks the assumption that the parameter value is in the interior of its parameter space. The null distribution is unknown in general and precise methods therefore requires numerical methods. Use of the χ^2 distribution with the usual dfs tends to inflate the p-values. Thus you can be confident in significant findings. More time consuming bootstrapping methods are needed for small, not significant p-values. However, careful coding using parallel methods can greatly quicken these computations.
- **Expected mean squares**: These are more powerful than the LRT but they often require tedious algebra that is dependent on the model under study. They also require extensive adjustments for unbalanced designs.
- **Testing fixed effects**: For testing the fixed effects we must use unrestricted maximum likelihood estimation. One can use the usual LRT, but this test may be anti-conservative. The parametric bootstrap is recommended here.

Example: Brightness of Paper Pulp Depending on Shift Operator (continued)

In this example the only fixed effect is the mean and there is not much interest in testing that. We first fit the null model

```
## null model fit
nullmod = lm(bright ~ 1, pulp)

## alternative model fit
smod = lmer(bright ~ 1 + (1|operator), pulp, REML = FALSE)
```

As there are no random effects in the null model we must use `lm`. For models of the same class, we could use built in software to perform a statistical test. Here we have to do the testing manually

```
as.numeric(2*(logLik(smod) - logLik(nullmod)))
```

```
## [1] 2.568371
```

```
pchisq(2.568371, df = 1, lower = FALSE)
```

```
## [1] 0.1090199
```

The p-value for this test is well above a nominal 5% significance level (assuming that testing is desired at this level). The use of the χ^2 test with its noted shortcomings allows for some doubt in this finding.

We will now try the parametric bootstrap procedure to obtain a more accurate p-value. We need to estimate the probability, given the null hypothesis is true, of observation an LRT of 2.568371 or greater. We have that

$$y \overset{H_0}{\sim} N(\mu, \sigma^2).$$

A simulation approach would generate data under this model, fit the null and alternative models and then compute the LRT. The process would be repeated a large number of times and the proportion of estimated LRTs exceeding the the observed LRT value of 2.568371 would be used to estimate the p-value. In practice we do not know the true values of μ and σ, but we can use plug-in.

Let's try it out. Fix the bootstrap sample size to be $B = 1000$ and then continue with the parametric bootstrap procedure

```
set.seed(13)
B = 1000
lrtstat = numeric(B)
```

```
system.time(for(b in 1:B){
  y = unlist(simulate(nullmod))
  bnull = lm(y ~ 1)
  balt = lmer(y ~ 1 + (1|operator), pulp, REML = FALSE)
  lrtstat[b] = as.numeric(2*(logLik(balt) - logLik(bnull)))
})
```

```
##    user  system elapsed
##   6.454   0.178   6.629
```

We may examine the distribution of the bootstrapped LRTs. We first compute the proportion that are close to zero

```
mean(lrtstat < 1e-5)
```

```
## [1] 0.69
```

The LRT clearly does not have a χ^2 distribution. See Section 8.2 in Faraway [2016] for discussion on this. An interesting refinement can be seen **here**. The parametric bootstrap may be the simplest approach, and the method we have used above is transparent and could be computed much more efficiently if needed. With all of this in mind the estimated p-value is

```
## p-value
pval = mean(lrtstat > 2.568371)
pval
```

```
## [1] 0.026
```

```
## simple standard error of the above
sqrt(pval*(1-pval)/B)
```

```
## [1] 0.005032296
```

We can be fairly sure that the estimated p-value is under a 0.05 nominal level. If in doubt, do some more replications to make sure; this only costs computer time (One can perform the above analysis with $B = 1e4$ and make it as fast as possible using parallel programming with either `mclapply` and/or `foreach` and whatever accompanying software packages are needed.) As it happens, this p-value is close to the fixed effects p-value.

In this example, the random and fixed effect tests gave similar outcomes. However, the hypotheses in random and fixed effects are intrinsically different. The conclusions from testing hypotheses involving fixed effects are only relevant to the levels under study. On the other hand, tests of hypotheses involving random effects are relevant of levels of the effect not considered. They are based upon a random generative mechanism that, in this case, are indexed by a single parameter. The conclusions of such test are more meaningful, more general, and should require more care than their fixed effects counterparts.

Predicting Random Effects

In a model with random effects, the α_i's are not fixed parameters to be estimated, but rather realizations from a random generative process. Under the standard normality assumption,

$$\alpha_i \sim N(0, \sigma_\alpha^2). \tag{1}$$

While we cannot estimate the α_i's in the classical sense, it is often of practical interest to predict their realized values. For instance, in the pulp example, we may wish to know which operators produce unusually bright or dim output, even if those specific operators are not the primary focus of inference. One approach is to summarize the random effect distribution itself through probabilities or expected values, but since $E(\alpha_i) = 0$ for all i by construction, this is of limited use. A more informative approach is to compute the *conditional* expectation $E(\alpha_i \mid Y)$, which combines the data with the prior distribution on α_i to produce individual predictions. These are known as best linear unbiased predictors (BLUPs).

If however, one approaches this problem from a Bayesian perspective then we have a prior density for the α's and $E(\alpha_i) = 0$ is just the prior mean. Let f represent a density function, then the posterior density for α is given by

$$f(\alpha_i|Y) \propto f(Y|\alpha_i)f(\alpha_i)$$

we can then find the posterior mean, denoted as $\hat{\alpha}$ as

$$E(\alpha_i|Y) = \int \alpha_i f(\alpha_i|Y)d\alpha_i.$$

Because both the prior on α_i and the likelihood are normal, this is a conjugate problem and the posterior mean has a closed form. Intuitively, the result is a *shrinkage estimator*: a weighted average between the data-driven signal about α_i and the prior mean of zero. When the random-effect variance σ^2_α is large relative to the residual variance σ^2, the data are trusted more and less shrinkage occurs. When σ^2_α is small, the α_i's are pulled strongly toward zero. Working through the conjugate normal calculations, this gives

$$\hat{\alpha} = DZ^TV^{-1}(Y - X\beta), \tag{2}$$

where:

- $D = \text{Var}(b)/\sigma^2$,
- Z is the fixed random-effects model matrix,
- $V = I + ZDZ^T$,
- $Y - X\beta$ are the residuals corresponding to a regression on the fixed-effects.

A purely Bayesian approach would specify the parameters of the prior (or specify priors for these) and compute a posterior distribution for α. We can also take an empirical Bayesian point of view and substitute the MLEs into D, V, and β to obtain predicted random-effects. This is the approach implemented in `lme4` for obtaining $\hat{\alpha}$ in (2):

```
ranef(mmod)$operator
```

```
##   (Intercept)
## a  -0.1219403
## b  -0.2591231
## c   0.1676679
## d   0.2133955
```

The predicted random effects are related to the fixed effects. We can show these for all operators

```
(cc = model.tables(lmod))
```

```
## Tables of effects
##
##  operator
## operator
##     a     b     c     d
## -0.16 -0.34  0.22  0.28
```

and then compute the ratio to the random effects as

```
## estimated fixed effects divided by predicted random effects
cc[[1]]$operator / ranef(mmod)$operator
```

```
##   (Intercept)
## a    1.312117
## b    1.312117
## c    1.312117
## d    1.312117
```

We see that the predicted random effects are exactly in proportion to the fixed effects. Typically, the predicted random effects are smaller and are viewed as a type of shrinkage estimate. The 95% confidence intervals for the random effects can be calculated and are displayed below:

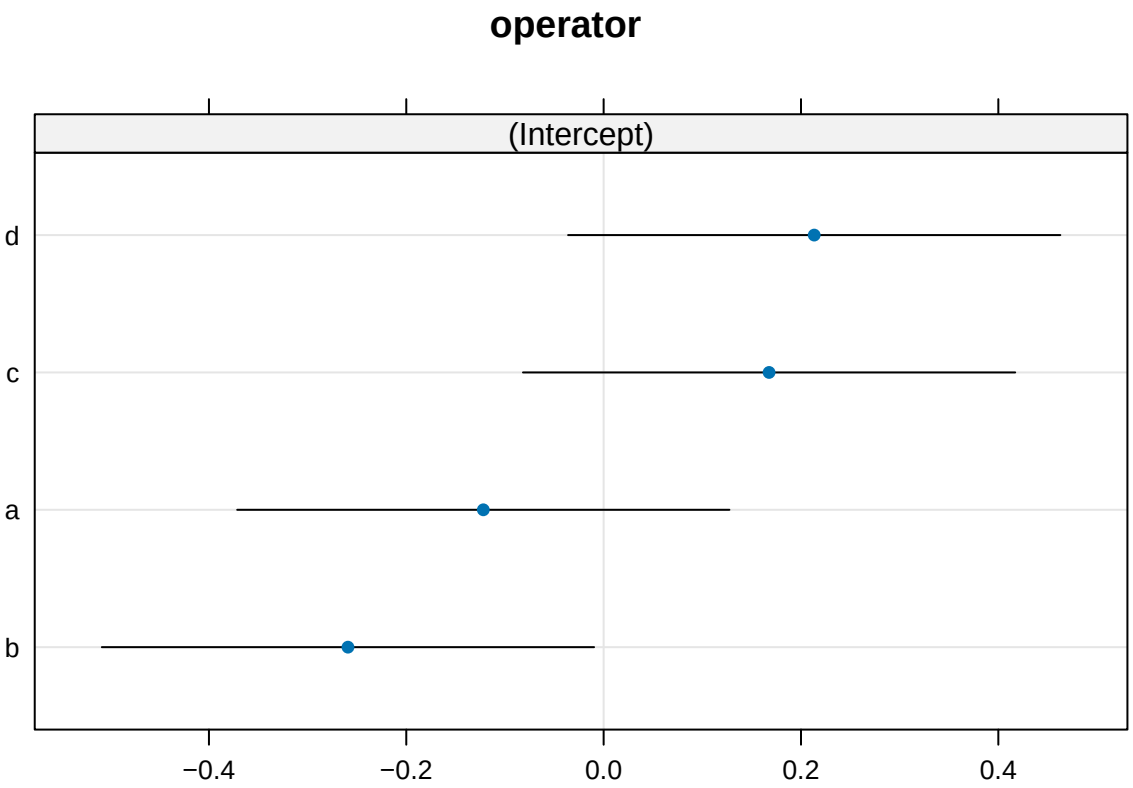
```
library(lattice)
```

```
##
## Attaching package: 'lattice'
```

```
## The following object is masked from 'package:faraway':
##
##     melanoma
```

```
dotplot(ranef(mmod, condVar=TRUE))
```

```
## $operator
```



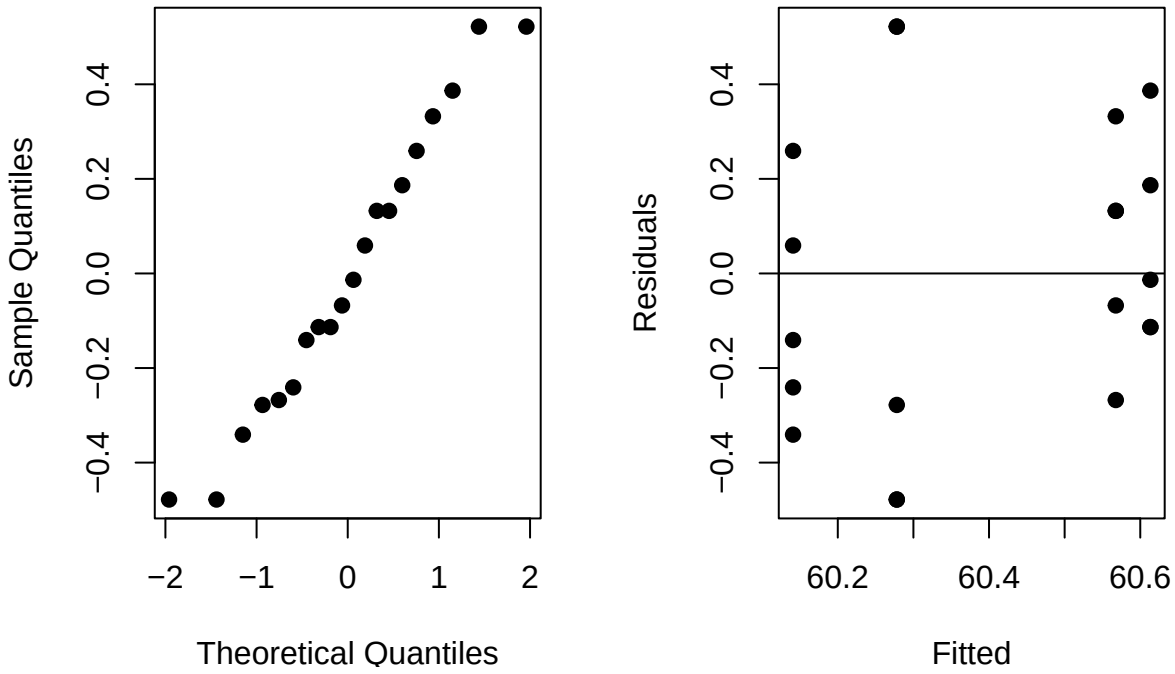
Suppose we wish to predict a new value. If the prediction is to be made for a new operator or unknown operator, the best we can do is give $\hat{\mu} = 60.4$. If we know the operator, then we can combine this with our fixed effects to give the best linear unbiased predictors (BLUPs) as follows

```
fixef(mmod) + ranef(mmod)$operator
```

```
##   (Intercept)
## a    60.27806
## b    60.14088
## c    60.56767
## d    60.61340
```

The implication of this is that we have more than one type of residual depending on what fitted values we use for inference. The default predicted values and residuals are from the outermost level of grouping. Thus, the usual diagnostic plots are still worthwhile.

```
par(mfrow = c(1,2))
qqnorm(resid(mmod), main="", pch = 19)
plot(fitted(mmod),resid(mmod), xlab="Fitted", ylab="Residuals", pch = 19)
abline(0,0)
```



Returning to the random effect predictions, we use a parametric bootstrap to compute standard errors for the predicted random effects. As in previous bootstrap procedures, the first step is to simulate responses from the fitted model, refit the model on the simulated data, and record the predicted values. However, there are two additional sources of variation to account for: variation due to sampling a new operator from the operator distribution, and variation due to a new observation from that operator. To capture both, we add normal noise with standard deviations equal to the estimated variance components. Note that if the goal is a confidence interval for the mean prediction rather than a prediction interval for a new observation, these extra noise terms should be omitted. We repeat this process 1000 times and take the 2.5th and 97.5th quantiles to obtain a 95% interval. We begin with the case of a new, previously unobserved operator:

```
group.sd = as.data.frame(VarCorr(mmod))$sdcor[1]
resid.sd = as.data.frame(VarCorr(mmod))$sdcor[2]
B = 1000
pv = numeric(B)
system.time(for(i in 1:B){
  y = unlist(simulate(mmod, use.u = TRUE))
  bmod = suppressWarnings(refit(mmod, y))
  pv[i] = predict(bmod, re.form=~0)[1] + rnorm(n=1,sd=group.sd) +  rnorm(n=1,sd=resid.sd)
})
```

```
##    user  system elapsed
##   6.344   0.030   6.374
```

```
quantile(pv, c(0.025, 0.975))
```

```
##     2.5%    97.5%
## 59.62780 61.24264
```

Some modification is necessary if we know the operator we are making the prediction interval for. We use the option `use.u=TRUE` in the simulate function indicating that we should simulate new values conditional on the estimated random effects. We need to do this because otherwise we would simulate an entirely new 'a' effect in each replication. Instead, we want to preserve the originally generated 'a' effect.

```
system.time(for(i in 1:B){
 y = unlist(simulate(mmod, use.u=TRUE))
  bmod = suppressWarnings(refit(mmod, y))
  pv[i] = predict(bmod, newdata=data.frame(operator="a")) + rnorm(n=1,sd=resid.sd)
})
```

```
##    user  system elapsed
##   9.136   0.056   9.192
```

```
quantile(pv, c(0.025, 0.975))
```

```
##     2.5%    97.5%
## 59.60897 60.97931
```

Split Plot Designs

Split plot designs originated in agriculture, but also occur in many other settings. A split plot design is as follows:

- One starts with a main whole plot in which a level of a factor variable is applied.
- The main plot is then divided into sections (split plots) and the levels of another factor variable is applied to each of these split plots.

The design arises as a result of restrictions on full randomization. For example, consider an agricultural experiment that studies the effects of plant varieties and irrigation techniques. It may be difficult to vary irrigation techniques over a field while it is easy to plant different plant varieties across a field. To overcome this challenge we consider a split plot design in which a sample of main plots (fields) each receive one irrigation technique at random. Each main plot is then divided into a number of split plots equal to the number of levels of plant varieties under study. Then each plant variety is randomly assigned to each split plot at random. Generally speaking, split plot designs arise when one factor is easy to change and another factor takes much more effort to manipulate. Here is a simple conceptual visual for split plot designs.

Example: Irrigation Split Plot Design

We now perform a data analysis corresponding to the motivating irrigation example. In this example we want to determine which irrigation and plant variety combination produces the highest yield. There are four irrigation methods and two plant varieties under consideration. Eight fields were available, but only one type of irrigation may be applied to each field. The whole plot factor is the irrigation method which should be randomly assigned to each of the fields. The fields may be divided into two halves with each plant variety randomly assigned to each of the halves. A summary and visualization of the data are included below:

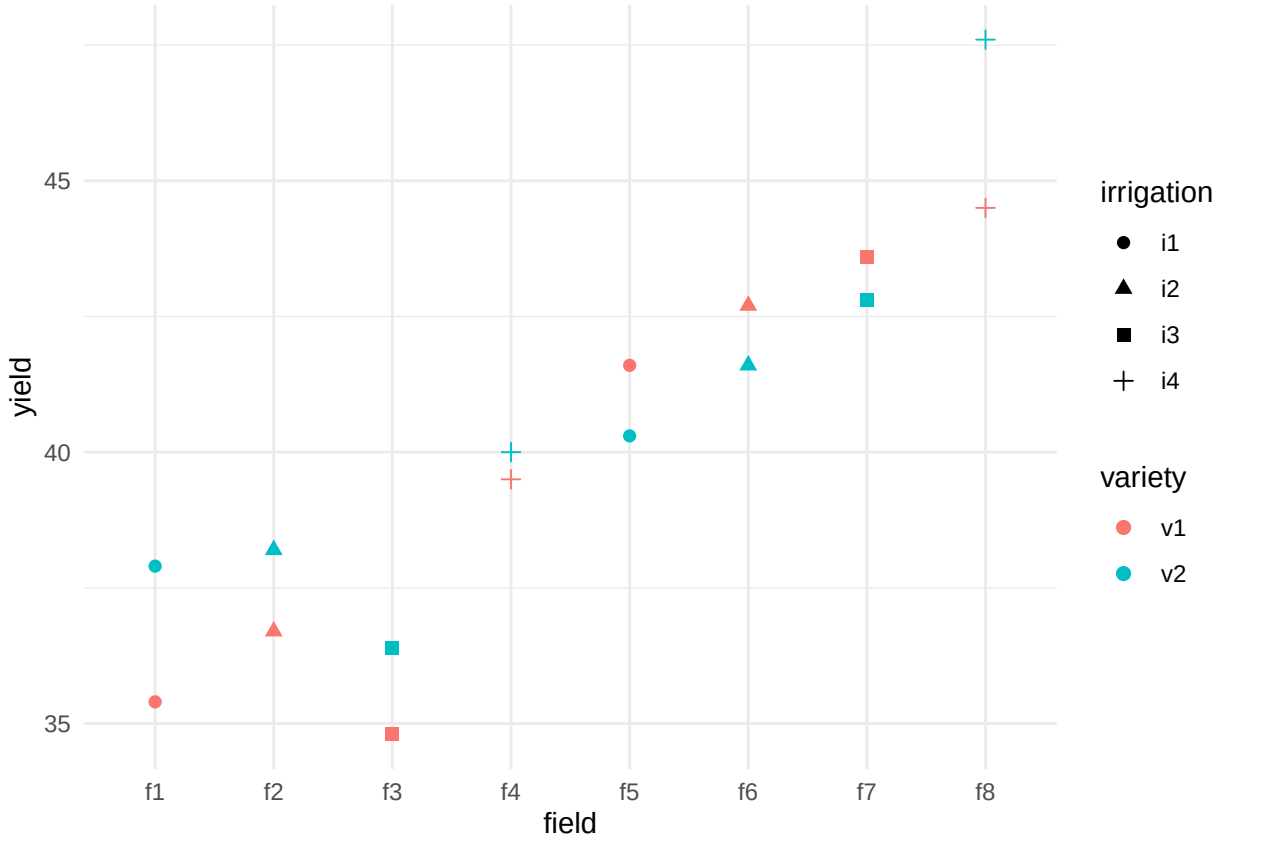
```
data(irrigation)

## data summary
summary(irrigation)
```

```
##      field   irrigation variety     yield
##  f1     :2   i1:4       v1:8    Min.   :34.80
##  f2     :2   i2:4       v2:8    1st Qu.:37.60
##  f3     :2   i3:4               Median :40.15
##  f4     :2   i4:4               Mean   :40.23
```

```
##  f5     :2                     3rd Qu.:42.73
##  f6     :2                     Max.   :47.60
##  (Other):4
```

```
## data visualization
ggplot(irrigation, aes(y=yield, x=field, shape=irrigation, color= variety)) +
  geom_point(size = 2) +
  theme_minimal()
```



Both irrigation and plant variety are fixed effects, but field should be modeled as a random effect (**Why so?**). We can also consider the interaction between field and variety, which is necessarily also a random effect because one of the two components is random. The fullest model that we might consider is:

$$y_{ijk} = \mu + r_i + v_j + (rv)_{ij} + f_k + (vf)_{jk} + \varepsilon_{ijk},$$

where

- μ is a fixed model intercept,
- r_i is the fixed effect for irrigation that ranges over $i \in \{1, \ldots, 4\}$ levels,
- v_j is the fixed effect for plant variety that ranges over $j \in \{1, 2\}$ levels,
- $(rv)_{ij}$ is the fixed effect interaction between irrigation and plant variety with an index ranging over i and j,
- f_k is the random effect for field for which there are 8 realizations each indexed by k,
- $(vf)_{jk}$ is the random effect interaction between plant variety and field with an index ranging over j and k, and
- ε is an error term for each observation.

We assume normal distributions for the random effect terms which are themselves indexed by single variance parameters σ_f^2, σ_{vf}^2, and σ_ε^2. Note that there is no irrigation and field interaction terms in this model. This is because it would not be possible to estimate such an effect since only one type of irrigation method is assigned to each field; these factors are not crossed. We try to fit this model as follows:

```
lmod = lmer(yield ~ irrigation * variety + (1|field) + (1|field:variety),
            data = irrigation)
```

However, when you try to fit such a model, it will fail because it is not possible to distinguish the variety within field variation. We would need more than one observation per variety within each field for us to separate the two variabilities. We are forced to resort to a simpler model that omits the variety by field interaction random effect:

$$y_{ijk} = \mu + r_i + v_j + (rv)_{ij} + f_k + \varepsilon_{ijk}$$

```
lmod = lmer(yield ~ irrigation * variety + (1|field),
            data = irrigation)
sumary(lmod)
```

```
## Fixed Effects:
##                       coef.est coef.se
## (Intercept)           40.23     1.47
## irrigation1           -1.43     2.54
## irrigation2           -0.42     2.54
## irrigation3           -0.82     2.54
## variety1              -0.37     0.36
## irrigation1:variety1   0.08     0.63
## irrigation2:variety1   0.27     0.63
## irrigation3:variety1   0.18     0.63
##
## Random Effects:
##  Groups   Name        Std.Dev.
##  field    (Intercept) 4.02
##  Residual             1.45
## ---
## number of obs: 16, groups: field, 8
## AIC = 76.5, DIC = 80.7
## deviance = 68.6
```

Note that `lmer` does not provide p-values for fixed-effects. Per Ben Bolker, one of the developers of `lmer`, not printing p-values has been a "feature" of `lmer` forever. For further discussion, see this formal technical FAQ and Doug Bates's informal and colorful explanation.

While this omission may seem surprising, it is a deliberate design choice rooted in the nature of mixed models. In classical linear models, p-values are derived from F-tests with clearly defined numerator and denominator degrees of freedom, based on a known distribution of a residual variance estimate. However, in mixed models the residual variance is estimated via REML, and its distribution is not analytically tractable—particularly when random effects are present and variance components are themselves estimated.

Although `lmer` uses a shared penalized residual variance across all fixed-effect tests, some software (like SAS) assigns different denominator degrees of freedom to each test using approximations (e.g., Satterthwaite or Kenward-Roger). Doug Bates, the author of `lmer`, considers this approach conceptually inconsistent: if the denominator variance is common across tests and its distribution is not known, assigning different degrees of freedom per test may give a false sense of precision. As a result, `lmer` omits p-values altogether. One can make model comparisons with formal inference available through companion tools like `pbkrtest`.

Returning to our analysis, we can see that the largest variance component is due to the field effect, with $\hat{\sigma}_f = 4.03$ compared to $\hat{\sigma}_\varepsilon = 1.45$. The relatively large standard errors, in comparison to the fixed effect

estimates, suggest that there may be no significant fixed effects. We can check this sequentially using a backwards model selection procedure with Kenward-Roger approximate F-tests that employ adjusted degrees of freedom and are implemented in the previously mentioned `pbkrtest` package.

```
library(pbkrtest)
lmoda = lmer(yield ~ irrigation + variety + (1|field), data=irrigation)
KRmodcomp(lmod, lmoda)
```

```
## large : yield ~ irrigation * variety + (1 | field)
## small : yield ~ irrigation + variety + (1 | field)
##         stat    ndf    ddf F.scaling p.value
## Ftest 0.2452 3.0000 4.0000         1  0.8612
```

We find there is no significant interaction term. We can now test each of the main effects starting with the fixed effect for plant variety:

```
lmodi = lmer(yield ~ irrigation + (1|field), irrigation)
KRmodcomp(lmoda, lmodi)
```

```
## large : yield ~ irrigation + variety + (1 | field)
## small : yield ~ irrigation + (1 | field)
##         stat    ndf    ddf F.scaling p.value
## Ftest 1.5782 1.0000 7.0000         1  0.2493
```

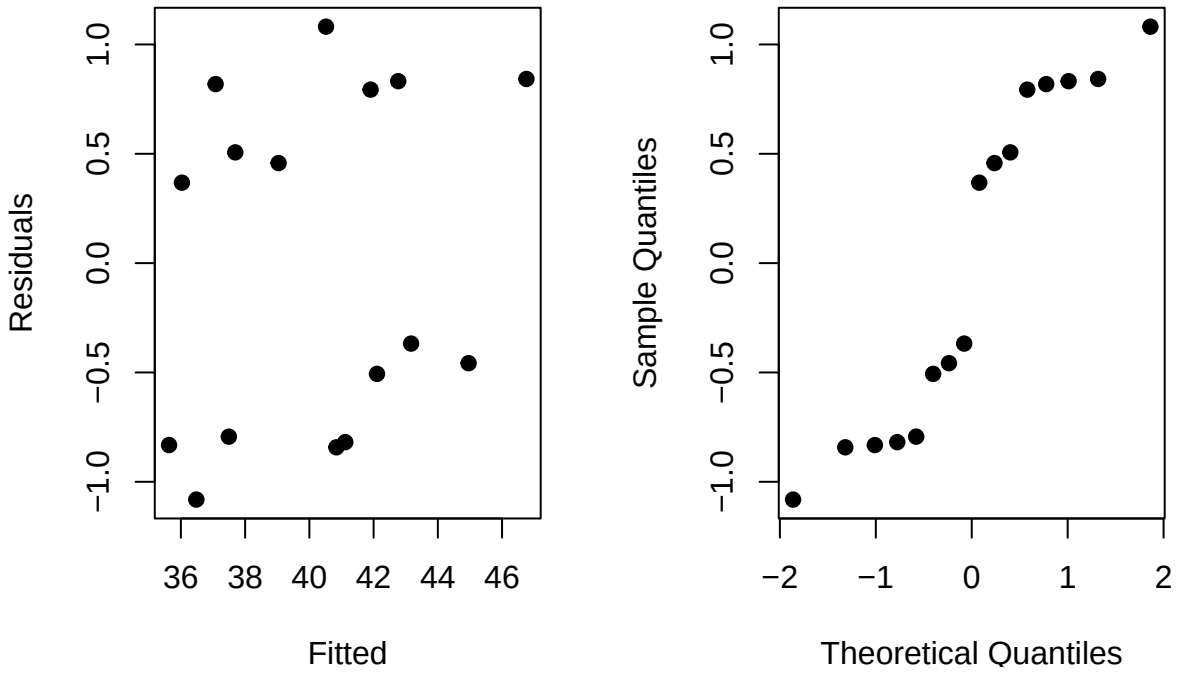
Dropping variety from the model seems reasonable since the p-value of 0.25 is large. We can test irrigation in a similar manner:

```
lmodv = lmer(yield ~ variety + (1|field), irrigation)
KRmodcomp(lmoda, lmodv)
```

```
## large : yield ~ irrigation + variety + (1 | field)
## small : yield ~ variety + (1 | field)
##         stat    ndf    ddf F.scaling p.value
## Ftest 0.3882 3.0000 4.0000         1  0.7685
```

Irrigation also fails to be significant. We should check the diagnostic plots to make sure there is nothing amiss:

```
par(mfrow = c(1,2))
plot(fitted(lmod), residuals(lmod), xlab="Fitted", ylab="Residuals", pch = 19)
qqnorm(residuals(lmod), main="", pch = 19)
```



We can see that there is no problem with the nonconstant variance, but that the residuals indicate a bimodal distribution caused by the pairs of observations in each field. This type of divergence from normality is unlikely to cause any major problems with the estimation and inference. We can test the random effects using the `exactRLRT` function which performs a likelihood ratio test based on simulated values from the finite sample distribution for testing whether the variance of a random effect is 0 in a linear mixed model with known correlation structure of the tested random effect and iid errors:

```
library(RLRsim)
exactRLRT(lmod)
```

```
##
##  simulated finite sample distribution of RLRT.
##
##  (p-value based on 10000 simulated values)
##
## data:
## RLRT = 6.1118, p-value = 0.0094
```

We see that yields does seem to vary across fields as the result is significant.

Example: Modeling Photoprotection Dynamics in Soybean

This example is a mixed-effects modeling demonstration adapted from a recent study on improving the efficiency of crop photosynthesis via genetic manipulation to speed up the relaxation of photoprotective mechanisms during sun-to-shade transitions.

In this demonstration, the researchers aim to identify genotypes that differ in their expression of apparent quantum efficiency (AqE) relative to a reference genotype (RC). AqE is a component of the photosynthetic process, and improvements in AqE can potentially lead to increased yield. The dataset includes repeated measurements across multiple plots, days, and two field seasons. Plot effects are treated as nuisance variation.

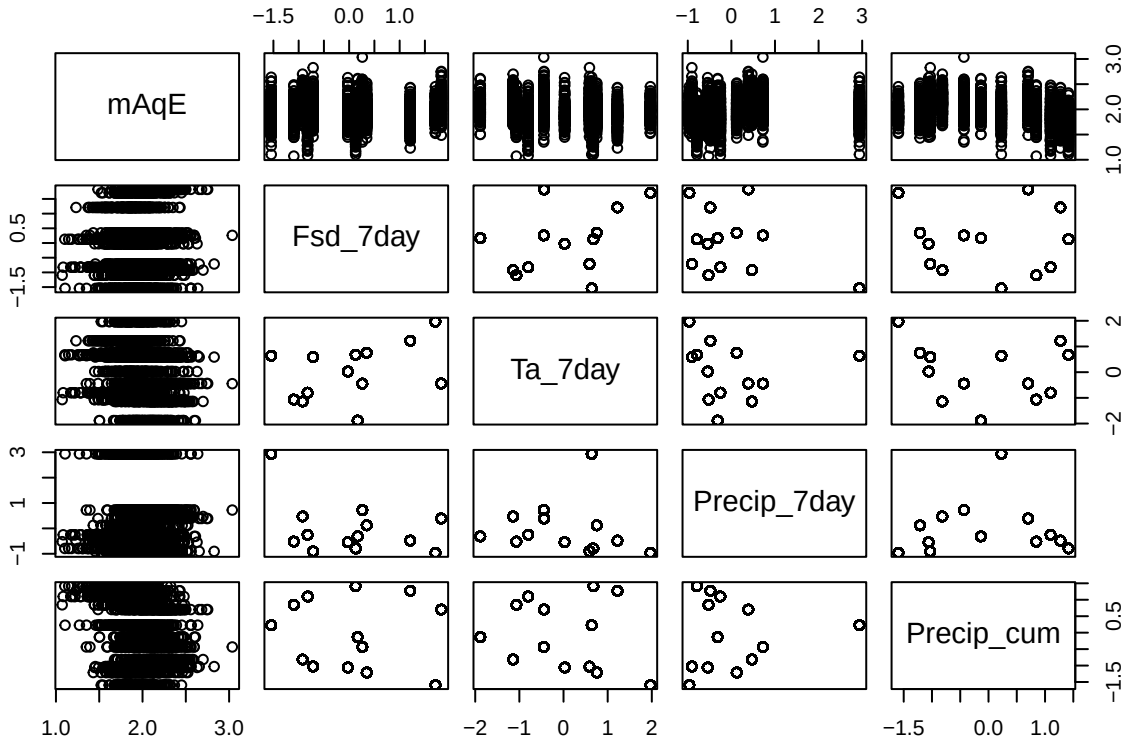
It is known that AqE exhibits a quadratic trend over the growing season (date), and may also depend on environmental conditions. Several weather-related covariates are available: (Fsd_7day, Ta_7day, Precip_7day, and Precip_cum).

```
library(tidyverse)
dat = read_csv("soybean_course.csv") %>%
  select(-mAqI, -mtqE, -mAqM, -mtqM, -maxNPQ, -Ta, -VPD, -Precip,
         -Fsd, -VPD_7day)
dat$ID = as.factor(dat$ID)
head(dat)
```

```
## # A tibble: 6 x 12
##   plot_number Date       ID     mAqE Ta_7day Precip_7day Fsd_7day Precip_cum
##         <dbl> <date>     <fct> <dbl>   <dbl>       <dbl>    <dbl>      <dbl>
## 1           1 2021-06-24 NAM46  2.59 -1.14         0.470  -0.923      -0.816
## 2           1 2021-06-30 NAM46  2.45  0.639        2.94   -1.54        0.227
## 3           1 2022-07-12 NAM26  2.30  0.753        0.122   0.347      -1.21
## 4           1 2021-07-13 NAM46  2.01 -1.07        -0.523  -1.10         0.845
## 5           1 2022-07-19 NAM26  1.98  0.0248      -0.544  -0.0269     -1.06
## 6           1 2021-07-20 NAM46  1.78 -0.800       -0.253  -0.823       1.10
## # i 4 more variables: year <dbl>, plot_number_year <chr>, Date_day <dbl>,
## #   Date_num <dbl>
```

This data consists of multiple measurements on the same day with only a few days of data collection. Here is a depiction of the relationships between the response and the weather variables.

```
pairs(dat %>% select(mAqE, Fsd_7day, Ta_7day, Precip_7day, Precip_cum))
```



We will now fit and compare models under three circumstances:

1. model fit to all data with plot random effects
2. model fit to 2021 data with plot random effects
3. model fit to 2022 data with plot random effects

We will consider two functional forms for weather variables and date variables:

1. quadratic model in date only

2. quadratic model in date with main effect terms for additional weather variables (denoted full model)

This gives a total of six candidate models fit in a common order:

1. m1_mAqE
2. m1_mAqE_full
3. m2_mAqE
4. m2_mAqE_full
5. m3_mAqE
6. m3_mAqE_full

The goal of this modeling framework is to identify genotypes whose expression of AqE differs meaningfully from the reference line (RC). Our procedure was designed to balance principled statistical modeling with a highly selective threshold for declaring genotype influence, reflecting the level of caution and robustness the researchers deemed necessary for biological interpretation.

Our selection pipeline proceeds as follows:

1. Investigate model diagnostics for the six candidate models. Refit with transformations if needed.
2. Determine if each genotype provides a better fit over reference level through a leave-one-variable-out AIC procedure. We refit the model with the genotype under investigation removed and record AIC values for this model and the original model with the genotype included. The models are fit so that the genotype effect is interpreted as its difference from the RC reference level (the RC level is encoded in the model intercept).
3. Obtain coefficient estimates for the genotype under study.
4. Report genotypes that have consistent sign estimates and are determined to differentiate from the reference level across all six candidate models (ie which genotypes improves model fit under this model class). These are the genotypes that we call influential.

```
## full data
m1_mAqE = lmer(mAqE ~ ID + Date_num + I(Date_num^2) +
                 (1|plot_number_year),
               data = dat, REML = FALSE,
               control = lmerControl(optimizer ="Nelder_Mead"))
m1_mAqE_full = lmer(mAqE ~ ID + Date_num + I(Date_num^2) +
                      Fsd_7day + Ta_7day + Precip_7day + Precip_cum +
                      (1|plot_number_year),
                    data = dat, REML = FALSE,
                    control = lmerControl(optimizer ="Nelder_Mead"))

## year 2021 data
m2_mAqE = lmer(mAqE ~ ID + Date_num + I(Date_num^2) +
                 (1|plot_number),
               data = dat %>% filter(year == 2021), REML = FALSE,
               control = lmerControl(optimizer ="Nelder_Mead"))
m2_mAqE_full = lmer(mAqE ~ ID + Date_num + I(Date_num^2) +
                      Fsd_7day + Ta_7day + Precip_7day + Precip_cum +
                      (1|plot_number),
                    data = dat %>% filter(year == 2021), REML = FALSE,
                    control = lmerControl(optimizer ="Nelder_Mead"))

## year 2022 data
m3_mAqE = lmer(mAqE ~ ID + Date_num + I(Date_num^2) +
                 (1|plot_number),
               data = dat %>% filter(year == 2022), REML = FALSE,
               control = lmerControl(optimizer ="Nelder_Mead"))
m3_mAqE_full = lmer(mAqE ~ ID + Date_num + I(Date_num^2) +
```

```
                      Fsd_7day + Ta_7day + Precip_7day + Precip_cum +
                      (1|plot_number),
                    data = dat %>% filter(year == 2022), REML = FALSE,
                    control = lmerControl(optimizer ="Nelder_Mead"))
```

Just as before we can use a parametric bootstrap to compare nested models. In this case we compare the linear regression model to a mixed-effects model containing a random effect for plots. In this example we can see that the parametric bootstrap is prohibitively time-consuming.

```
set.seed(13)
B = 1000
lrtstat = numeric(B)

# parametric bootstrap
system.time(for(b in 1:B){
  y = unlist(simulate(m1_mAqE))
  bnull = lm(y ~ ID + Date_num + I(Date_num^2), data = dat)
  balt = lmer(y ~ ID + Date_num + I(Date_num^2) +
    (1|plot_number_year),
    data = dat, REML = FALSE,
    control = lmerControl(optimizer ="Nelder_Mead"))
  lrtstat[b] = as.numeric(2*(logLik(balt) - logLik(bnull)))
})
```

```
##    user  system elapsed
##  77.493   1.876  79.421
```

We examine the distribution of the bootstrapped LRTs. We observe a large mass at zero and only observe a few non-zero bootstrap observations.

```
mean(lrtstat < 1e-5)
```

```
## [1] 0
```

The bootstrapped p-value is 0.

```
## p-value
pval = mean(lrtstat >
  as.numeric(2*(logLik(m1_mAqE_full) - logLik(m1_mAqE))))
pval
```

```
## [1] 0
```

We now compare the mixed-effects model containing a random effect for plots with a larger mixed-effects model containing fixed effects for weather variables. Given the computational cost of the parametric bootstrap, we could proceed with model selection criteria such as AIC and BIC, which in this case strongly support the more complex model with random effects for plots and fixed effects for weather variables.

```
## AIC
AIC(m1_mAqE, m1_mAqE_full)
```

```
##              df        AIC
## m1_mAqE      45  -51.25128
## m1_mAqE_full 49 -295.66744
```

```
AIC(m2_mAqE, m2_mAqE_full)
```

```
##              df         AIC
## m2_mAqE      45    7.947379
```

```
## m2_mAqE_full 49 -165.855423
```

```
AIC(m3_mAqE, m3_mAqE_full)
```

```
##              df       AIC
## m3_mAqE      45 -304.2494
## m3_mAqE_full 48 -350.7777
```

```
## BIC
BIC(m1_mAqE, m1_mAqE_full)
```

```
##              df       BIC
## m1_mAqE      45 207.48762
## m1_mAqE_full 49 -13.92952
```

```
BIC(m2_mAqE, m2_mAqE_full)
```

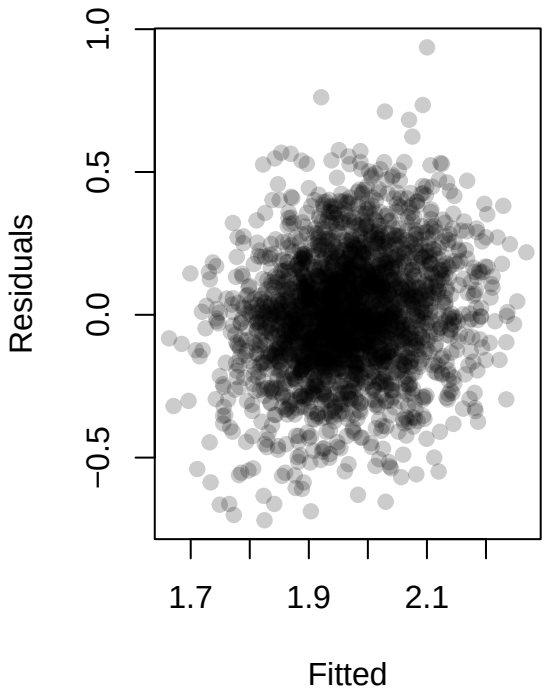
```
##              df       BIC
## m2_mAqE      45 240.46408
## m2_mAqE_full 49  87.32943
```

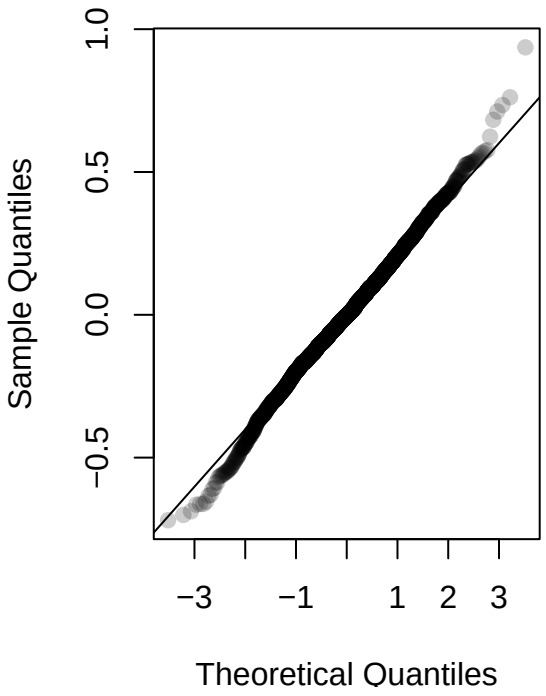
```
BIC(m3_mAqE, m3_mAqE_full)
```

```
##              df        BIC
## m3_mAqE      45  -82.28925
## m3_mAqE_full 48 -114.02023
```

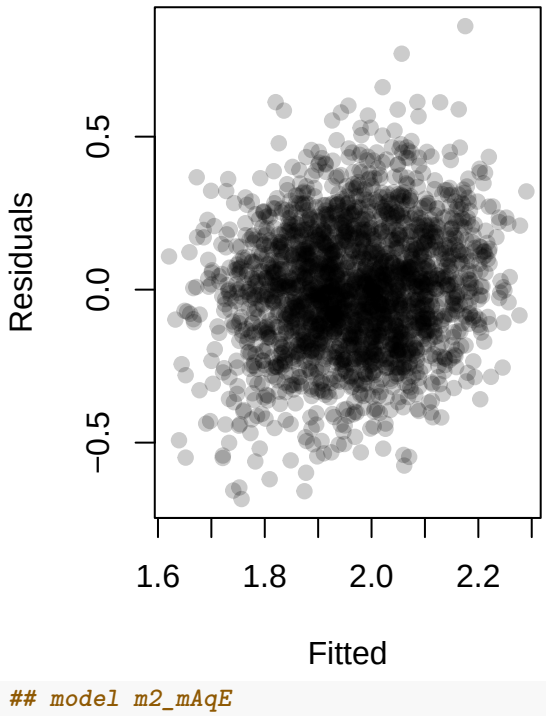
Below we see that the modeling assumptions of constant variance and normality of residuals are satisfied in all candidate models.

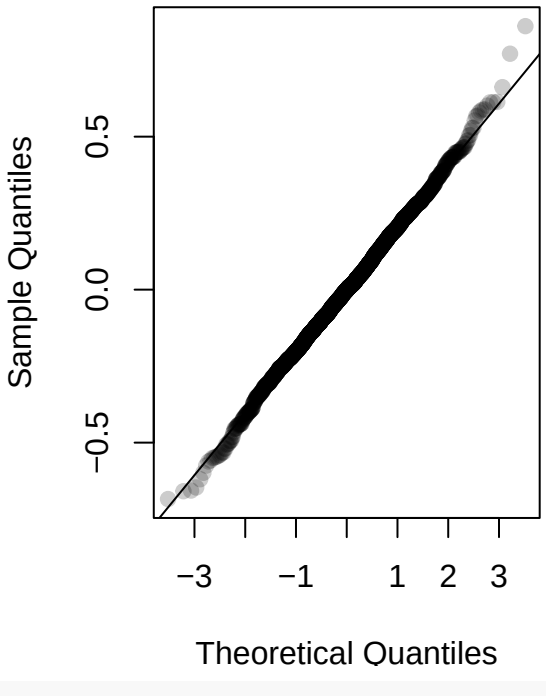
```
## model m1_mAqE
par(mfrow = c(1,2))
plot(fitted(m1_mAqE), residuals(m1_mAqE),
     xlab="Fitted", ylab="Residuals", pch = 19,
     col = rgb(0,0,0,alpha=0.2))
a = qqnorm(residuals(m1_mAqE), main="", pch = 19,
           col = rgb(0,0,0,alpha=0.2))
qqline(residuals(m1_mAqE))
```





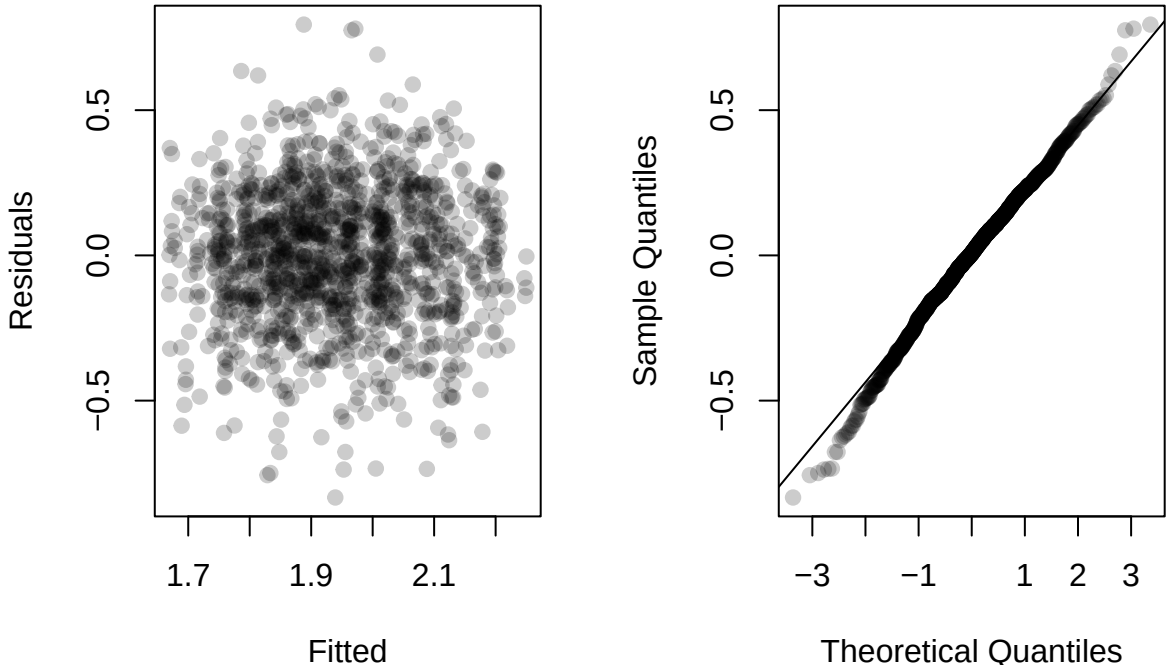

```
## model m1_mAqE_full
par(mfrow = c(1,2))
plot(fitted(m1_mAqE_full), residuals(m1_mAqE_full),
     xlab="Fitted", ylab="Residuals", pch = 19,
     col = rgb(0,0,0,alpha=0.2))
a = qqnorm(residuals(m1_mAqE_full), main="", pch = 19,
           col = rgb(0,0,0,alpha=0.2))
qqline(residuals(m1_mAqE_full))
```



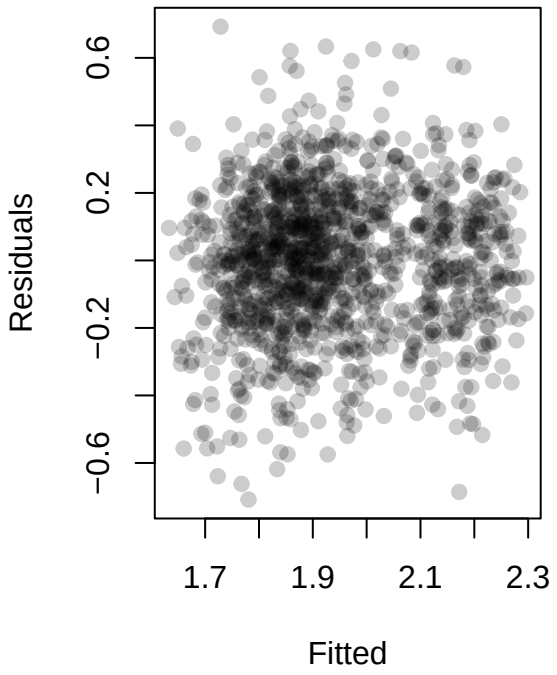


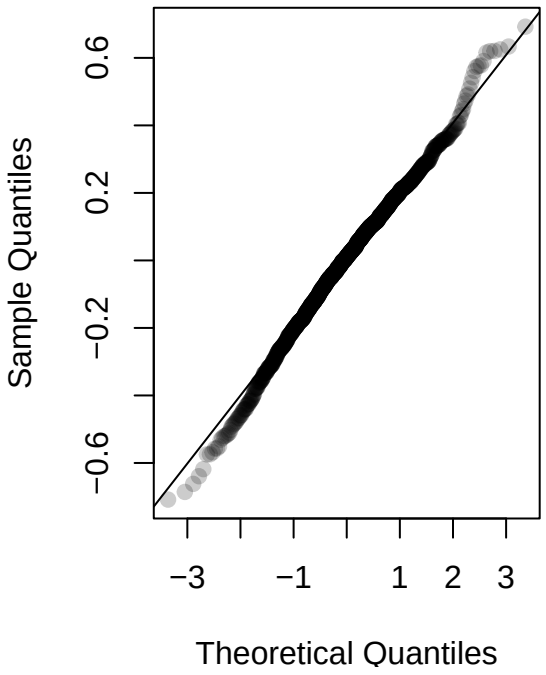

```
## model m2_mAqE
par(mfrow = c(1,2))
```

```r
plot(fitted(m2_mAqE), residuals(m2_mAqE),
     xlab="Fitted", ylab="Residuals", pch = 19,
     col = rgb(0,0,0,alpha=0.2))
a = qqnorm(residuals(m2_mAqE), main="", pch = 19,
           col = rgb(0,0,0,alpha=0.2))
qqline(residuals(m2_mAqE))
```

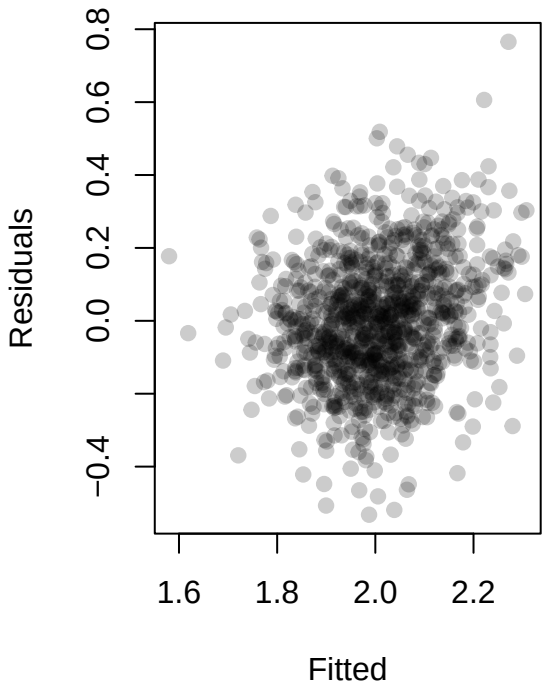


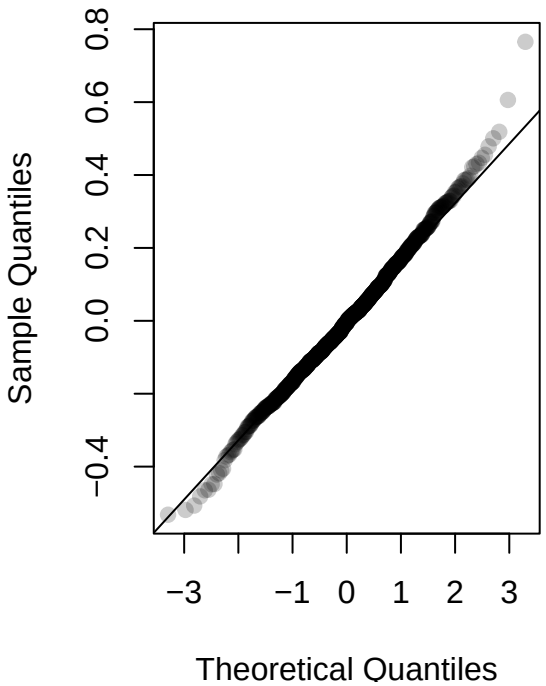

```r
## m2_mAqE_full
par(mfrow = c(1,2))
plot(fitted(m2_mAqE_full), residuals(m2_mAqE_full),
     xlab="Fitted", ylab="Residuals", pch = 19,
     col = rgb(0,0,0,alpha=0.2))
a = qqnorm(residuals(m2_mAqE_full), main="", pch = 19,
           col = rgb(0,0,0,alpha=0.2))
qqline(residuals(m2_mAqE_full))
```





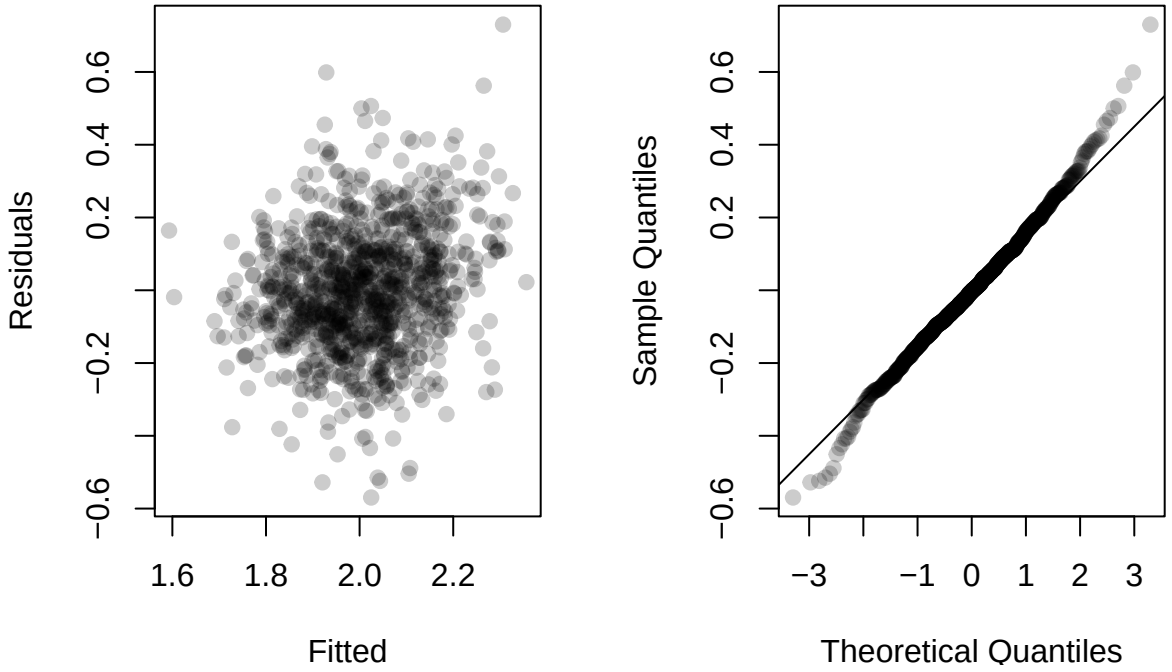

```
## model m3_mAqE
par(mfrow = c(1,2))
plot(fitted(m3_mAqE), residuals(m3_mAqE),
     xlab="Fitted", ylab="Residuals", pch = 19,
     col = rgb(0,0,0,alpha=0.2))
a = qqnorm(residuals(m3_mAqE), main="", pch = 19,
           col = rgb(0,0,0,alpha=0.2))
qqline(residuals(m3_mAqE))
```






```
## model m3_mAqE_full
par(mfrow = c(1,2))
```

```r
plot(fitted(m3_mAqE_full), residuals(m3_mAqE_full),
     xlab="Fitted", ylab="Residuals", pch = 19,
     col = rgb(0,0,0,alpha=0.2))
a = qqnorm(residuals(m3_mAqE_full), main="", pch = 19,
           col = rgb(0,0,0,alpha=0.2))
qqline(residuals(m3_mAqE_full))
```



We now investigate which genotypes are associated with AqE values that are different than the RC reference level. To do this we will construct a model with a particular genotype level removed, and then compare the smaller model with the full model selected above. Model comparisons will be made using AIC. We will perform this procedure for each genotype. Results are displayed below, negative values indicate that a genotype is associated with AqE values that are different than the RC reference level.

```r
## set up model matrices for all models
M = model.matrix(mAqE ~ ID + Date_num + I(Date_num^2), data = dat)
M_full = model.matrix(mAqE ~ ID + Date_num + I(Date_num^2) +
                        Fsd_7day + Ta_7day + Precip_7day + Precip_cum,
                        data = dat)

M2021 = model.matrix(mAqE ~ ID + Date_num + I(Date_num^2),
                       data = dat %>% filter(year == 2021))
M2021_full = model.matrix(mAqE ~ ID + Date_num + I(Date_num^2) +
                            Fsd_7day + Ta_7day + Precip_7day + Precip_cum,
                            data = dat %>% filter(year == 2021))

M2022 = model.matrix(mAqE ~ ID + Date_num + I(Date_num^2),
                       data = dat %>% filter(year == 2022))
M2022_full = model.matrix(mAqE ~ ID + Date_num + I(Date_num^2) +
                              Fsd_7day + Ta_7day + Precip_7day + Precip_cum,
                            data = dat %>% filter(year == 2022))

## AIC for each ID variable from full AqE fixed-effects model
library(parallel)
```

```
ncores = detectCores() - 2
AIC_IDs_mAqE = do.call(rbind, mclapply(
  grep("ID", colnames(M)), function(j){
    M1 = M[, -j]
    foo = lmer(mAqE ~ -1 + M1 + (1|plot_number_year),
               data = dat, REML = FALSE)
    M1_full = M_full[, -j]
    foo_full = lmer(mAqE ~ -1 + M1_full + (1|plot_number_year),
                    data = dat, REML = FALSE)
    M12021 = M2021[, -j]
    bar = lmer(mAqE ~ -1 + M12021 + (1|plot_number),
               data = dat %>% filter(year == 2021), REML = FALSE)
    M12021_full = M2021_full[, -j]
    bar_full = lmer(mAqE ~ -1 + M12021_full + (1|plot_number),
                    data = dat %>% filter(year == 2021), REML = FALSE)
    M12022 = M2022[, -j]
    baz = lmer(mAqE ~ -1 + M12022 + (1|plot_number),
               data = dat %>% filter(year == 2022), REML = FALSE)
    M12022_full = M2022_full[, -j]
    baz_full = lmer(mAqE ~ -1 + M12022_full + (1|plot_number),
                    data = dat %>% filter(year == 2022), REML = FALSE)

    c(AIC(m1_mAqE) - AIC(foo),
      AIC(m1_mAqE_full) - AIC(foo_full),
      AIC(m2_mAqE) - AIC(bar),
      AIC(m2_mAqE_full) - AIC(bar_full),
      AIC(m3_mAqE) - AIC(baz),
      AIC(m3_mAqE_full) - AIC(baz_full))
  }, mc.cores = ncores))
rownames(AIC_IDs_mAqE) = colnames(M)[grep("ID", colnames(M))]
```

We now report influential genotypes according to our selection procedure.

```
coefs_mAqE = round(cbind(summary(m1_mAqE)$coefficients[2:41, 1],
                         summary(m1_mAqE_full)$coefficients[2:41, 1],
                         summary(m2_mAqE)$coefficients[2:41, 1],
                         summary(m2_mAqE_full)$coefficients[2:41, 1],
                         summary(m3_mAqE)$coefficients[2:41, 1],
                         summary(m3_mAqE_full)$coefficients[2:41, 1]), 3)
inference_mAqE = sign(coefs_mAqE) * sign(AIC_IDs_mAqE < 0)

## influential genotypes
rownames(coefs_mAqE)[abs(rowSums(inference_mAqE)) == 6]
```

```
## [1] "ID13" "ID17" "ID26" "ID27" "ID32"
```

```
coefs_mAqE[abs(rowSums(inference_mAqE)) == 6, ]
```

```
##        [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
## ID13  0.122  0.119  0.127  0.126  0.113  0.114
## ID17 -0.138 -0.137 -0.120 -0.123 -0.158 -0.159
## ID26 -0.089 -0.090 -0.099 -0.100 -0.083 -0.089
## ID27 -0.086 -0.084 -0.077 -0.075 -0.093 -0.091
## ID32  0.078  0.079  0.074  0.076  0.093  0.093
```

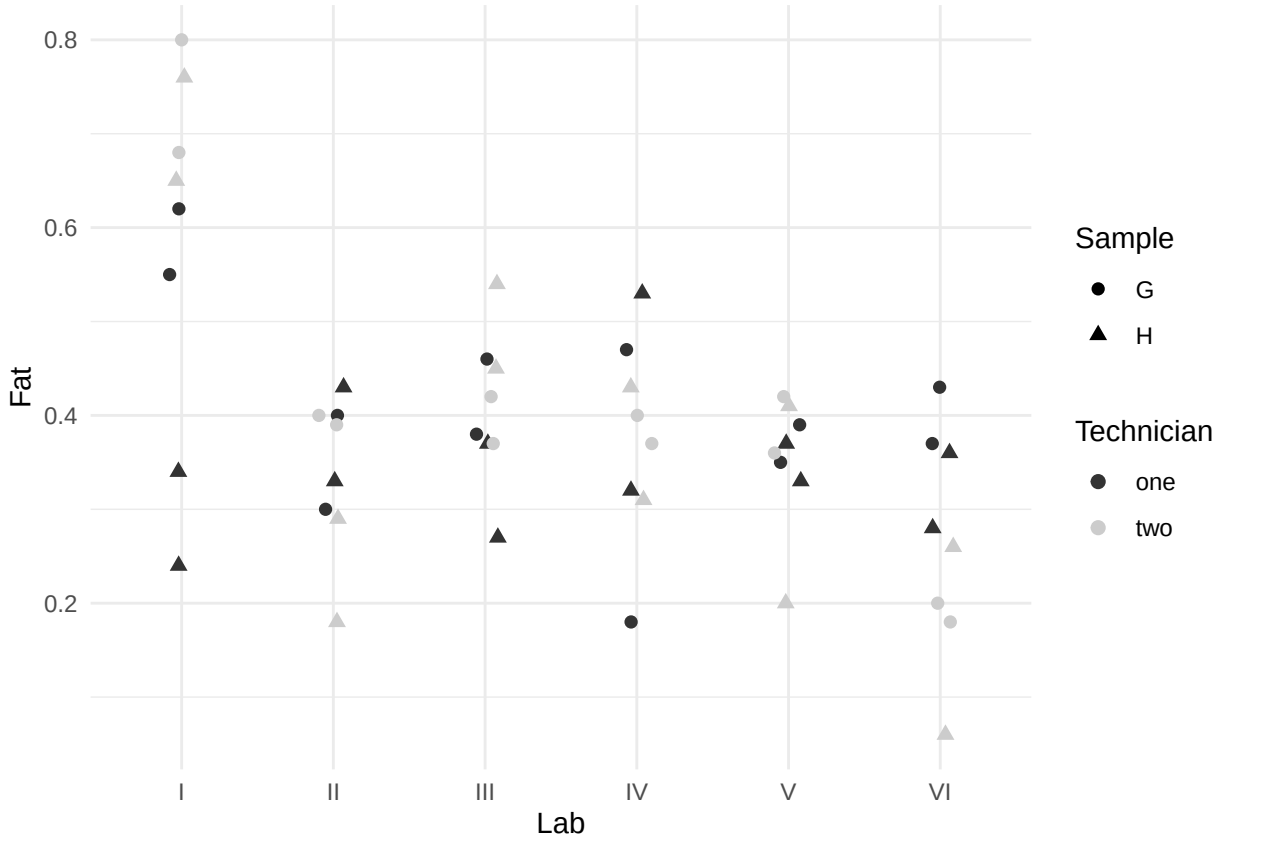
Example: Nested Random Effects in a Laboratory Setting

In this section we provide an example of nesting. Consistency between laboratory tests is important and yet the results may depend on who did the test and where the test was performed. In an experiment to test levels of consistency, a large jar of homogenized dried egg powder was divided up into a number of samples. Because the powder was homogenized, the fat content of the samples is the same, but this fact is withheld from the laboratories. Four samples were sent to each of six laboratories. Two of the samples were labeled as G and two as H, although in fact they were identical. The laboratories were instructed to give two samples to two different technicians. The technicians were then instructed to divide their samples into two parts and measure the fat content of each. So each laboratory reported eight measures, each technician four measures, that is, two replicated measures on each of two samples. The data comes from Bliss (1967):

```
data(eggs)
summary(eggs)
```

```
##       Fat          Lab    Technician Sample
##  Min.   :0.0600   I  :8   one:24     G:24
##  1st Qu.:0.3075   II :8   two:24     H:24
##  Median :0.3700   III:8
##  Mean   :0.3875   IV :8
##  3rd Qu.:0.4300   V  :8
##  Max.   :0.8000   VI :8
```

```
ggplot(eggs, aes(y=Fat, x=Lab, color=Technician, shape=Sample)) +
  geom_point(size = 2, position = position_jitter(width=0.1, height=0.0)) +
  scale_color_grey() +
  theme_minimal()
```



Note that although the technicians have been labeled "one" and "two," they are two different people in each

lab. Thus the technician factor is nested within laboratories. Furthermore, note that even though the samples are labeled "H" and "G," these are not the same samples across the technicians and the laboratories. Hence we have samples nested within technicians.

Technicians and samples should be treated as random effects since we may consider these as randomly sampled. If the labs were specifically selected, then they should be taken as fixed effects. If, however, they were randomly selected from those available, then they should be treated as random effects. If the purpose of the study is to come to some conclusion about consistency across laboratories, the latter approach is advisable. For the purposes of this analysis, we will treat labs as random. So all our effects (except the grand mean) are random. The model is:

$$y_{ijkl} = \mu + L_i + T_{ij} + S_{ijk} + \varepsilon_{ijkl}$$

where,

- μ is a fixed model intercept,
- L_i is the random effect for lab for which there are 6 realizations each indexed by i,
- T_{ij} is the random effect for technician nested within lab for which there are 2 realizations each indexed by j within each lab i,
- S_{ijk} is the random effect for sample nested within technician nested within lab. There are four realizations each indexed by k by each technician j within each lab i, and
- ε is an error term for each observation.

This can be fit with:

```
cmod = lmer(Fat ~ 1 + (1|Lab) + (1|Lab:Technician) +  (1|Lab:Technician:Sample),
            data=eggs)
sumary(cmod)
```

```
## Fixed Effects:
## coef.est  coef.se
##     0.39     0.04
##
## Random Effects:
##  Groups                Name        Std.Dev.
##  Lab:Technician:Sample (Intercept) 0.06
##  Lab:Technician        (Intercept) 0.08
##  Lab                   (Intercept) 0.08
##  Residual                          0.08
## ---
## number of obs: 48, groups: Lab:Technician:Sample, 24; Lab:Technician, 12; Lab, 6
## AIC = -54.2, DIC = -73.3
## deviance = -68.8
```

So we have that the estimated variance components are all similar in magnitude. The lack of consistency in measures of fat content can be ascribed to variance between labs, technicians, measurement due to different labeling, and measurement error.

Although the data has a natural hierarchical structure which suggests a particular order of testing, we might reasonably wonder which of the components contribute substantially to the overall variation. Why test the sample effect first? A look at the confidence intervals reveals the problem:

```
confint(cmod, method="boot")
```

```
##                  2.5 %     97.5 %
## .sig01      0.00000000 0.09564357
## .sig02      0.00000000 0.13957263
```

```
## .sig03      0.00000000 0.14948989
## .sigma      0.06081876 0.10674937
## (Intercept) 0.30653979 0.47883284
```

We might drop any of the three random effect terms but it is not possible to be sure which is best to go. It is safest to conclude there is some variation in the fat measurement coming from all three sources.

Repeated Measures and Longitudinal Data

In repeated measures designs there are several individuals (or units) under study and multiple measurements are taken repeatedly on each individual. When these repeated measurements are taken over time, it is called a longitudinal study or, in some applications, a panel study. Typically there are various covariates concerning the individual that are also recorded and interest centers on how the response depends on these covariates over time. Often it is reasonable to believe that the response of each individual has several components:

- a fixed effect, which is a function of the covariates;
- a random effect, which expresses the variation between individuals;
- and an error, which is due to measurement or unrecorded variables.

In a repeated measures design we will suppose that each individual has a response Y_i which is now a vector of length n_i and is modeled conditionally on the random effect b_i as

$$Y_i|b_i \sim N(X_i\beta + Z_ib_i, \sigma^2\Lambda_i).$$

Note that this is a very similar model used in the linear mixed-effects model construction above, with the exception that we now allow for the individuals to have a more general covariance structure Λ_i. As before, we will assume that $b_i \sim N(0, \sigma^2 D)$ so that

$$Y_i \sim N(X_i\beta, \Sigma_i)$$

where $\Sigma_i = \sigma^2(\Lambda_i + Z_iDZ_i^T)$. Now suppose that we have N individuals and assume that the measurement errors and random effects between individuals are uncorrelated. Then we can combine the data as:

$$Y = \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_N \end{bmatrix}, \qquad X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_N \end{bmatrix}, \qquad B = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_N \end{bmatrix},$$

and

- $\tilde{D} = \text{diag}(D, D, \ldots, D)$,
- $Z = \text{diag}(Z_1, Z_2, \ldots, Z_N)$,
- $\Sigma = \text{diag}(\Sigma_1, \Sigma_2, \ldots, \Sigma_N)$, and
- $\Lambda = \text{diag}(\Lambda_1, \Lambda_2, \ldots, \Lambda_N)$, where

$Y_i \in \mathbb{R}^{n_i}$, $X_i \in \mathbb{R}^{n_i \times p}$, $\beta \in \mathbb{R}^p$, $b_i \in \mathbb{R}^q$, $Z_i \in \mathbb{R}^{n_i \times q}$, and the rest of the quantities in the above follow from these specifications. With this setup we can write the model as

$$Y \sim N(X\beta, \Sigma) \qquad \text{where} \qquad \Sigma = \sigma^2(\Lambda + Z\tilde{D}Z^T).$$

The log-likelihood for the data is then computed as previously and estimation, testing, standard errors and confidence intervals all follow using standard likelihood theory as before. There is no strong distinction between repeated measures methodology and the methodology of linear mixed-effects models.

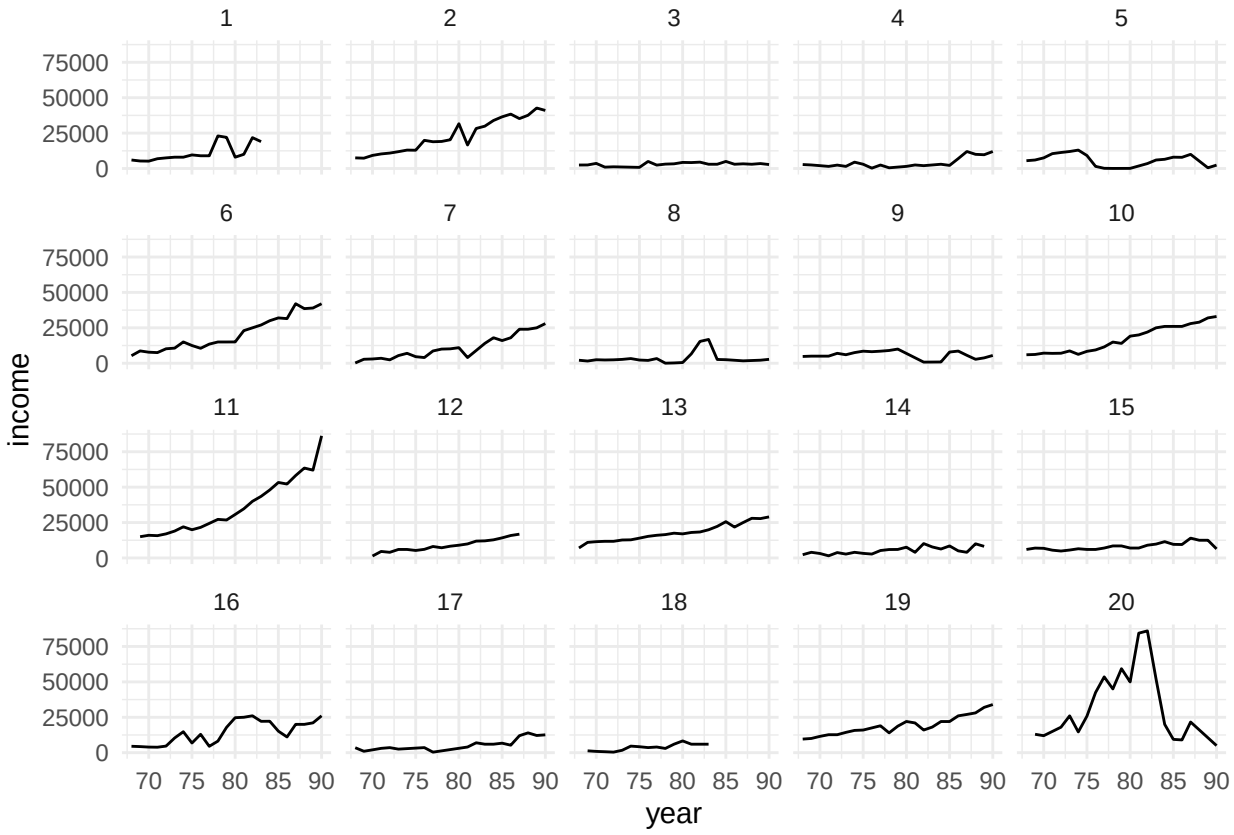
Example: Panel Study of Income Dynamics

The Panel Study of Income Dynamics (PSID), begun in 1968, is a longitudinal study of a representative sample of U.S. individuals described in Hill (1992). The study is conducted at the Survey Research Center, Institute for Social Research, University of Michigan, and is still continuing. There are currently 8700 households in the study and many variables are measured. We chose to analyze a random subset of this data, consisting of 85 heads of household who were aged 25–39 in 1968 and had complete data for at least 11 of the years between 1968 and 1990. The variables included were annual income, biological sex, years of education and age in 1968:

```
library(faraway)
data(psid)
```

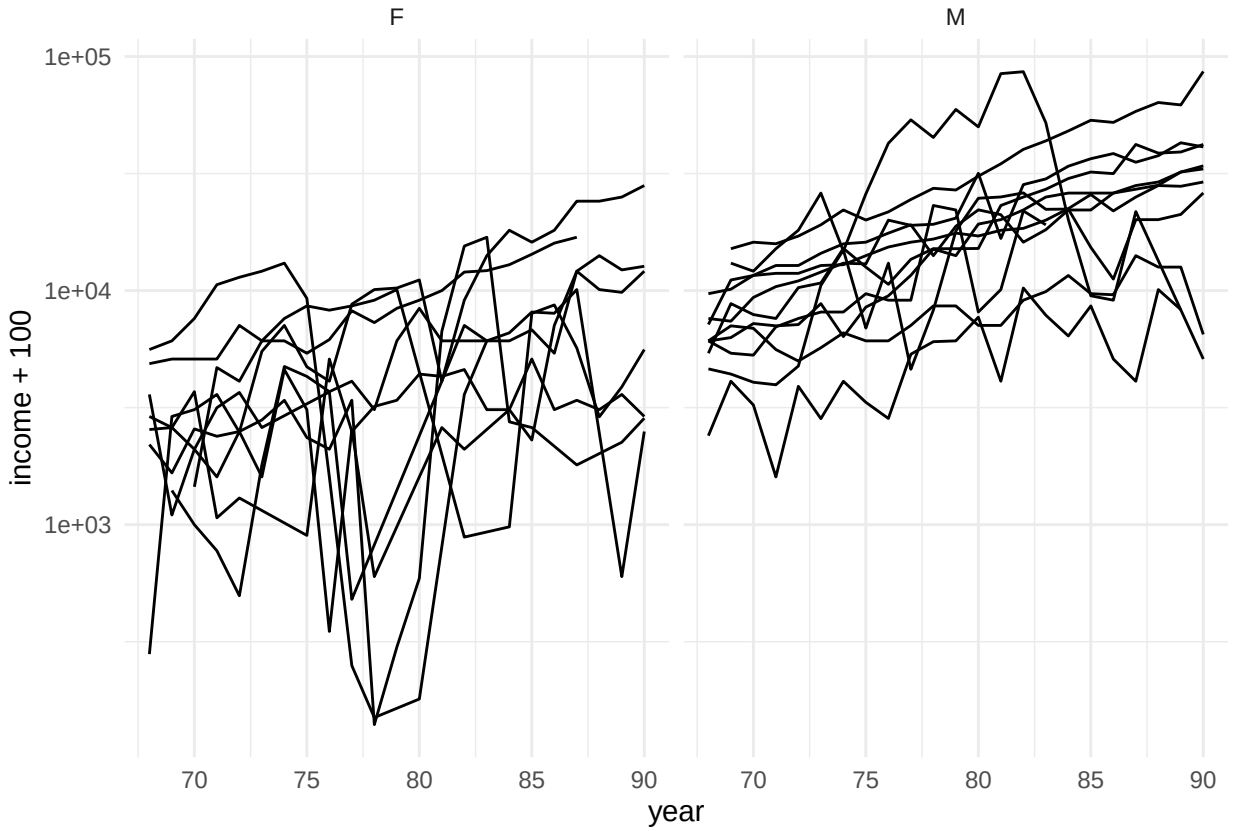
Now plot the data for the first 20 individuals:

```
psid20 = filter(psid, person <= 20)
ggplot(psid20, aes(x=year, y=income)) +
  geom_line() +
  facet_wrap(~ person) +
  theme_minimal()
```



We see that some individuals have a slowly increasing income, typical of someone in steady employment in the same job. Other individuals have more erratic incomes. We can also show how the incomes vary by sex. Income is more naturally considered on a log-scale:

```
ggplot(psid20, aes(x=year, y=income+100, group=person)) +
  geom_line() +
  facet_wrap(~ sex) +
  scale_y_log10() +
  theme_minimal()
```



We added \$100 to the income of each subject to remove the effect of some subjects having very low incomes for short periods of time. These cases distorted the plots without the adjustment. We see that men's incomes are generally higher and less variable while women's incomes are more variable, but are perhaps increasing more quickly. We could fit a line to each subject starting with the first:

```
lmod = lm(log(income) ~ I(year-78), subset=(person==1), psid)
coef(lmod)
```

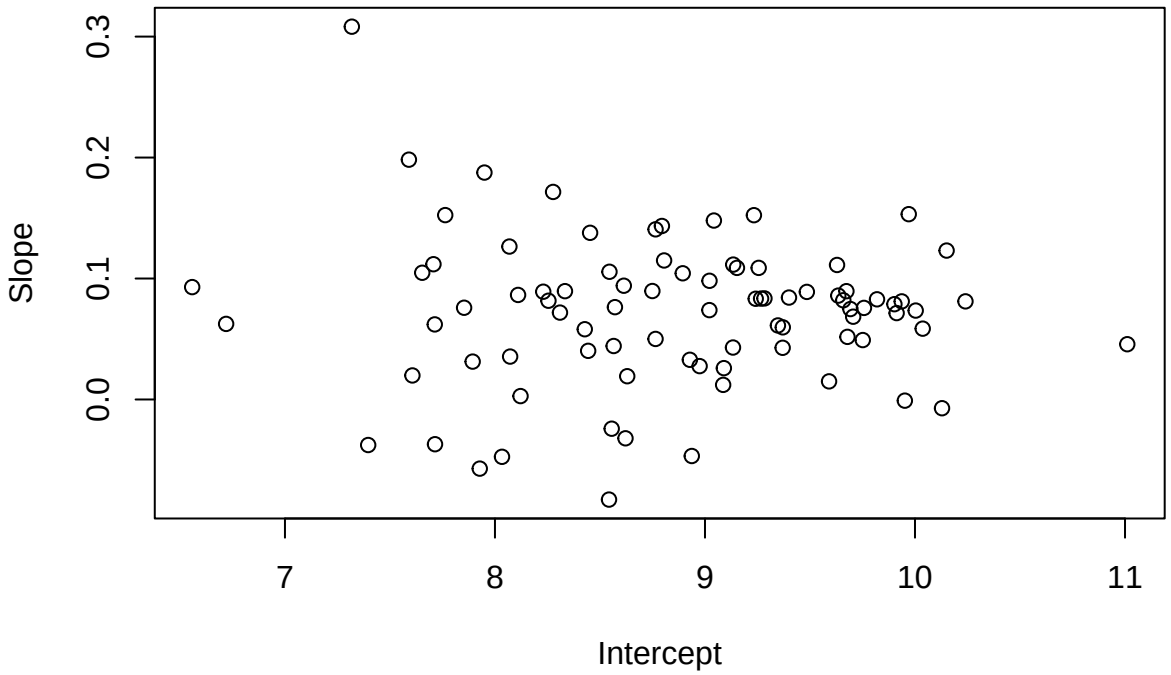
```
##  (Intercept) I(year - 78)
##    9.3999568    0.0842667
```

Note that we have centered the predictor year at its median value so that the intercept will represent the predicted log income in 1978 and not the year 1900 which is nonsense (any sensible location transformation would be appropriate, but centering with respect to the median is perhaps most appropriate). We now fit a line for all the subjects and plot the results:

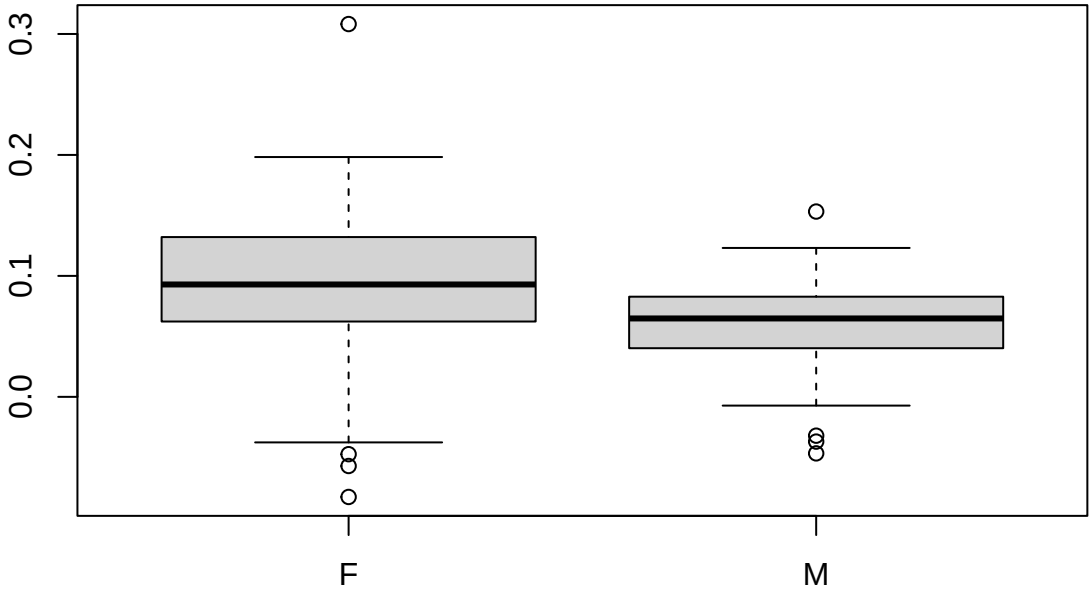
```
ml = lmList(log(income) ~ I(year-78) | person, psid)
intercepts = sapply(ml,coef)[1,]
slopes = sapply(ml,coef)[2,]
```

The `lmList` command fits a linear model to each group within the data, here specified by person. A list of linear models, one for each group, is returned from which we extract the intercepts and slopes.

```
plot(intercepts,slopes,xlab="Intercept",ylab="Slope")
```




```
psex = psid$sex[match(1:85,psid$person)]
boxplot(split(slopes,psex))
```



In the first panel, we see how the slopes relate to the intercepts — there is little correlation. This means we can test incomes and income growths separately. In the second panel, we compare the income growth rates where we see these as higher and more variable for women compared to men. We can test the difference in income growth rates for men and women:

```
t.test(slopes[psex=="M"],slopes[psex=="F"])
```

```
##
##  Welch Two Sample t-test
##
## data:  slopes[psex == "M"] and slopes[psex == "F"]
```

```
## t = -2.3786, df = 56.736, p-value = 0.02077
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  -0.05916871 -0.00507729
## sample estimates:
##  mean of x  mean of y
## 0.05691046 0.08903346
```

We see that women have a significantly higher growth rate than men. We can also compare the incomes at the intercept (which is 1978):

```
t.test(intercepts[psex=="M"],intercepts[psex=="F"])
```

```
##
##  Welch Two Sample t-test
##
## data:  intercepts[psex == "M"] and intercepts[psex == "F"]
## t = 8.2199, df = 79.719, p-value = 3.065e-12
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  0.8738792 1.4322218
## sample estimates:
## mean of x mean of y
##  9.382325  8.229275
```

We see that men have significantly higher incomes.

This is an example of a response feature analysis. It requires choosing an important characteristic. We have chosen two here: the slope and the intercept. For many datasets, this is not an easy choice and at least some information is lost by doing this. Response feature analysis is attractive because of its simplicity. By extracting a univariate response for each individual, we are able to use a wide array of well-known statistical techniques. However, it is not the most efficient use of the data as all the additional information besides the chosen response feature is discarded. Notice that having additional data on each subject would be of limited value.

Suppose that the income change over time can be partly predicted by the subject's age, sex and educational level. We do not expect a perfect fit as there are other factors that will affect a subject's income. These factors may cause the income to be generally higher or lower or they may cause the income to grow at a faster or slower rate. The variation may be partitioned into two components. We can model this variation with a random intercept and slope, respectively, for each subject. We also expect that there will be some year-to-year variation within each subject. For simplicity, let us initially assume that this error is homogeneous and uncorrelated, that is, $\Lambda_i = I$. We also center the year to aid interpretation as before. We may express these notions in the model:

```
psid$cyear = psid$year - 78
mmod = lmer(log(income) ~ cyear*sex + age + educ + (cyear|person), psid,
            control = lmerControl(optimizer = "bobyqa"))
```

This model can be written as:

$$\log(\text{income})_{ij} = \mu + \beta_y \text{year}_j + \beta_g \text{sex}_i + \beta_{yg} \text{sex}_i * \text{year}_j + \beta_e \text{education}_i + \beta_a \text{age}_i + \gamma_i^0 + \gamma_i^1 \text{year}_j + \varepsilon_{ij}$$

where i indexes the individuals and j indexes the years, and log is the natural logarithm. We have:

$$\begin{pmatrix} \gamma_j^0 \\ \gamma_j^1 \end{pmatrix} \sim N(0, \sigma^2 D)$$

The model summary is

```r
#summary(mmod, digits=3)
sumary(mmod, digits=3)
```

```
## Fixed Effects:
##             coef.est coef.se
## (Intercept)  7.249    0.540
## cyear        0.072    0.006
## sex1        -0.575    0.061
## age          0.011    0.014
## educ         0.104    0.021
## cyear:sex1   0.013    0.006
##
## Random Effects:
##  Groups   Name        Std.Dev. Corr
##  person   (Intercept) 0.531
##           cyear       0.049    0.187
##  Residual             0.684
## ---
## number of obs: 1661, groups: person, 85
## AIC = 3842.5, DIC = 3748.4
## deviance = 3785.5
```

We now analyze this summary table. Lets start with the fixed effects. We see that income increases about 10% for each additional year of education (exp(0.104)). We see that age does not appear to be significant. For females, the reference level in this example, income increases about 8.5% a year, while for men, it increases about 7.2% a year. We see that, for this data, the incomes of women in 1978 are exp(-0.575) = 0.56% that of men!

Now the random effects. We know the mean for males and females, but individuals will vary about this. The standard deviation for the intercept and slope are 0.531 and 0.049 ($\sigma\sqrt{D_{11}}$ and $\sigma\sqrt{D_{22}}$), respectively. These have a correlation of 0.187 ($\text{cor}(\gamma^0, \gamma^1)$). Finally, there is some additional variation in the measurement not so far accounted for having standard deviation of 0.684 ($\text{sd}(\varepsilon_{ij})$). We see that the variation in increase in income is relatively small while the variation in overall income between individuals is quite large. Furthermore, given the large residual variation, there is a large year-to-year variation in incomes.

How `lme4` fits Mixed-Effects Models

This Section will follow the package vignette corresponding to the `lme4` package. This vignette can be seen here. We will consider the simple mixed-effects structure

$$\begin{aligned} Y|b &\sim N(X\beta + Zb, \sigma^2 I), \\ b &\sim N(0, \Sigma), \end{aligned}$$

although it should be noted that the vignette considers a slightly more general formulation involving known mean offsets o and known variance weights W, and that much of the materials presented here are aimed at convenience, computational efficiency, and robustness.

It is convenient to express Σ as

$$\Sigma_\theta = \sigma^2 \Lambda_\theta \Lambda_\theta^T$$

where $\Lambda_\theta \in \mathbb{R}^{q \times q}$ is known as a *relative covariance factor* depending on a *variance-component parameter* vector θ.

The following table indexes the dimensions of linear mixed-effects models, the subscript $i = 1, \ldots, k$ denotes

a specific random-effects term:

Symbol	Size
n	Length of the response vector Y
p	Number of columns of fixed-effects model matrix X
k	Number of random-effects terms
p_i	Number of columns of the raw model matrix X_i
l_i	Number of levels of the grouping factor indices i_i
$q_i = p_i l_i$	Number of columns of the term-wise model matrix Z_i
$q = \sum_i^k q_i$	Number of columns of random-effects model matrix Z
$m_i = \binom{p_i+1}{2}$	Number of covariance parameters for term i
$m = \sum_i^k m_i$	Total number of covariance parameters

The following table denotes the symbols used to describe the structure of the random-effects model matrix and the relative covariance factor. The subscript $i = 1, \ldots, k$ denotes a specific random-effects term:

Symbol	Size	Description
X_i	$n \times p_i$	Raw random-effects model matrix
J_i	$n \times l_i$	Indicator matrix of grouping factor indices
X_{ij}	$p_i \times 1$	Column vector containing jth row of X_i
J_{ij}	$l_i \times 1$	Column vector containing jth row of J_i
i_i	n	Vector of grouping factor indices
Z_i	$n \times q_i$	Term-wise random-effects model matrix
θ	m	Number of covariance parameters
T_i	$p_i \times p_i$	Lower triangular template matrix
Λ_i	$q_i \times q_i$	Term-wise relative covariance factor

It is important to note that X_i is a raw-random effects matrix and **NOT** a subpart of the fixed-effects model matrix X. This is a notational choice used in the lme4 vignette.

Algebraic and Computational Accounting of Mixed-Model Input Formulas

The fixed-effects terms of a mixed-model formula are parsed to produce the fixed-effects model matrix, X, in the same way that the R `lm` function generates model matrices.

However, a mixed-model formula incorporates $k \geq 1$ random-effects terms $(r|f)$ as well. These k terms are used to produce the random-effects model matrix Z, and the structure of the relative covariance factor Λ_θ, which are matrices that typically have a sparse structure.

We now describe how one might construct these matrices from the random-effects terms, considering first a single term (written as $(r|f)$ in a formula for a linear mixed model) and then generalizing to multiple terms. The above tables summarize the matrices and vectors that determine the structure of Z and Λ_θ.

The expression r in $(r|f)$ is a linear model formula that evaluates to an R model matrix X_i of size $n \times p_i$, called the raw random-effects model matrix for term i. A term is said to be a *scalar* random-effects term when $p_i = 1$, otherwise it is *vector-valued*. For a simple, scalar random-effects term of the form $(1|f)$, X_i is the $n \times 1$ matrix of ones, which implies a random intercept model.

The expression f evaluates to an R factor, called the grouping factor, for the term. For the ith term, we represent this factor mathematically with a vector i_i of factor indices, which is an n-vector of values from $1, \ldots, l_i$ where l_i is the number of levels of the grouping factor variable f in the ith random effects term. Let J_i be the $n \times l_i$ matrix of indicator columns for i_i. Using the `Matrix` package (Bates and Maechler 2015) in R, we may construct the transpose of J_i from a factor vector, f, by coercing f to a 'sparseMatrix' object. For example,

```
library(Matrix)
# first argument is the number of levels
```

```
# second argument is the number of replications
(f = gl(3, 2))
```

```
## [1] 1 1 2 2 3 3
## Levels: 1 2 3
```

```
(Ji = t(as(f, Class = "sparseMatrix")))
```

```
## 6 x 3 sparse Matrix of class "dgCMatrix"
##      1 2 3
## [1,] 1 . .
## [2,] 1 . .
## [3,] . 1 .
## [4,] . 1 .
## [5,] . . 1
## [6,] . . 1
```

When $k > 1$ we order the random-effects terms so that $l_1 \geq l_2 \geq \cdots \geq l_k$; in general, this ordering reduces "fill-in" (i.e., the proportion of elements that are zero in the lower triangle of $\Lambda_\theta^T Z^T Z \Lambda_\theta + I$ but not in the lower triangle of its left Cholesky factor L_θ). This reduction in fill-in provides more efficient matrix operations within the penalized least squares algorithm described below.

Examples: The PSID example above considered $k = 1$ term `(cyear|person)` which corresponded to a random intercept and a random slope for an individual's income over time. In this example $X_i, i = k = 1$, with $p_i = 2$ corresponding to a model matrix consisting of an intercept and a column for years centered around 1978, the median value, and J_i provides indexes for each individual in the study. The Bliss (1967) example above concerning consistency of measurements considered $k = 3$ terms `(1|Lab)`,`(1|Lab:Technician)`, and `(1|Lab:Technician:Sample)`. Here, $X_i, i = 1, 2, 3$, with $p_i = 1, i = 1, 2, 3$, corresponding to intercept vectors and the J_i matrices consider separate grouping factors which reflect the nested random effects.

Constructing the Random-Effects Model Matrix

The ith random-effects term contributes $q_i = l_i p_i$ columns to the model matrix Z. These columns are grouped into a matrix Z_i, which is referred to as the *term-wise model matrix* for the ith term. Thus q, the number of columns in Z and the dimension of the random variable b is

$$q = \sum_{i=1}^{k} q_i = \sum_{i=1}^{k} l_i p_i.$$

Creating the matrix Z_i from X_i and J_i is straightforward but is somewhat awkward to describe. Consider Z_i as being further decomposed into l_i blocks of p_i columns. The rows in the first block are the rows of X_i multiplied by the 0/1 values in the first column of J_i and similarly for the subsequent blocks. With these definitions we may define the term-wise random-effects model matrix Z_i for the ith term as a transposed Khatri-Rao product,

$$Z_i = (J_i^T * X_i^T)^T = \begin{bmatrix} J_{i1}^T \otimes X_{i1}^T \\ J_{i2}^T \otimes X_{i2}^T \\ \vdots \\ J_{in}^T \otimes X_{in}^T \end{bmatrix},$$

where $*$ and $\otimes$ are, respectively, the Khatri-Rao and Kronecker products, and J_{ij}^T and X_{ij}^T are row vectors of the the jth rows of J_i and X_i. These rows correspond to the jth sample in the response vector Y and j runs from 1 to n. The `Matrix` package in R contains a `KhatriRao` function, which can be used to form Z_i. For example, if we begin with a raw model matrix,

```
(Xi = cbind(1, rep.int(c(-1, 1), 3L)))
```

```
##      [,1] [,2]
## [1,]    1   -1
## [2,]    1    1
## [3,]    1   -1
## [4,]    1    1
## [5,]    1   -1
## [6,]    1    1
```

```
(Zi = t(KhatriRao(t(Ji), t(Xi))))
```

```
## 6 x 6 sparse Matrix of class "dgCMatrix"
##
## [1,] 1 -1 .  . .  .
## [2,] 1  1 .  . .  .
## [3,] .  . 1 -1 .  .
## [4,] .  . 1  1 .  .
## [5,] .  . .  . 1 -1
## [6,] .  . .  . 1  1
```

In particular, for a simple, scalar term, Z_i is formed by element-wise multiplication of the single column of X_i by each of the columns of J_i. Because each Z_i is generated from indicator columns, its cross-product, $Z_i^T Z_i$ is block diagonal consisting of l_i diagonal blocks each of size p_i. Note that this means that when $k = 1$, $Z^T Z$ will be block diagonal. These block-diagonal properties allow for more efficient sparse matrix computations.

The full random-effects model matrix Z is constructed from $k \geq 1$ blocks,

$$Z = [Z_1 \; Z_2 \; \cdots \; Z_k].$$

A more detailed look at Z is gvien through its transpose where the columns represent samples and the rows represent random effect terms:

$$Z^T = \begin{bmatrix} J_{11} \otimes X_{11} & J_{12} \otimes X_{12} & \cdots & J_{1n} \otimes X_{1n} \\ J_{21} \otimes X_{21} & J_{22} \otimes X_{22} & \cdots & J_{2n} \otimes X_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ J_{k1} \otimes X_{k1} & J_{k2} \otimes X_{k2} & \cdots & J_{kn} \otimes X_{kn} \end{bmatrix}.$$

Note that the proportion of elements of Z^T that are structural zeros is

$$\frac{\sum_{i=1}^{k} p_i(l_i - 1)}{\sum_{i=1}^{k} p_i}.$$

Therefore, the sparsity of Z^T increases with the number of grouping factor levels. As the number of levels is often large in practice, it is essential for speed and efficiency to take account of this sparsity, for example by using sparse matrix methods, when fitting mixed models.

Constructing the Relative Covariance Factor

The $q \times q$ covariance factor Λ_θ is a block diagonal matrix whose ith diagonal block Λ_i is of size q_i, $i = 1, \ldots, k$. We refer to Λ_i as the *term-wise relative covariance factor*.

More details are given in the lme4 vignette.

Objective Function

In the initial parameterization of the linear mixed-effects model, the covariance parameter θ appears only in the marginal distribution of the random effects through the variance matrix Σ_θ. However, from the perspective of computational stability and efficiency, it is advantageous to reformulate the model such that

θ appears only in the conditional distribution for the response vector given the random effects. Such a reparameterization allows us to work with singular covariance matrices, which regularly arise in practice.

This reparameterization is made by defining a *spherical random-effects* variable U with distribution

$$U \sim N(0, \sigma^2 I_q).$$

If we set

$$b = \Lambda_\theta U,$$

then b will have the assumed marginal random-effects distribution in our setup. The conditional distribution of the response vector given the random effects may now be reformulated as,

$$Y|U = u \ \sim \ N(\mu_{Y|U=u}, \sigma^2),$$

where

$$\mu_{Y|U=u} = X\beta + Z\Lambda_\theta u$$

is a vector of linear predictors, which can be interpreted as a conditional mean (or mode in the case of normality). Similarly, we also define $\mu_{U|Y=y_{\text{obs}}}$ as the conditional mean (or mode in the case of normality) of the spherical random effects given the observed response vector. Note also that we use the u symbol throughout to represent a specific value of the random variable U.

Penalized Least Squares (PLS)

The computational methods for maximum likelihood estimation of the mixed-effects model involve repeated applications of the PLS method. In particular, the PLS problem is to minimize the penalized residual sum-of-squares,

$$r^2(\theta, \beta, u) = \rho^2(\theta, \beta, u) + \|u\|^2$$

over $[u^T, \beta^T]^T$, where

$$\rho^2(\theta, \beta, u) = \|y_{\text{obs}} - \mu_{Y|U=u}\|^2.$$

This notation makes explicit that the minimization depends on θ, β, and u. The penalization term $\|u\|^2$ penalizes models with larger magnitude values of u. In the so-called "pseudo-data" approach we write the penalized weighted residual sum-of-squares as the squared length of a block matrix equation,

$$r^2(\theta, \beta, u) = \left\| \begin{bmatrix} y_{\text{obs}} \\ 0 \end{bmatrix} - \begin{bmatrix} Z\Lambda_\theta & X \\ I_q & 0 \end{bmatrix} \begin{bmatrix} u \\ \beta \end{bmatrix} \right\|^2.$$

This pseudo-data approach shows that the PLS problem may also be thought of as a standard least squares problem for an extended response vector, which implies that the minimizing value $(\mu^T_{U|Y=y_{\text{obs}}}, \ \hat{\beta}^T_\theta)^T$ satisfies the normal equations,

$$\begin{bmatrix} \Lambda^T_\theta Z^T y_{\text{obs}} \\ X^T y_{\text{obs}} \end{bmatrix} = \begin{bmatrix} \Lambda^T_\theta Z^T Z \Lambda_\theta + I & \Lambda^T_\theta Z^T X \\ X^T Z \Lambda_\theta & X^T X \end{bmatrix} \begin{bmatrix} \mu_{U|Y=y_{\text{obs}}} \\ \hat{\beta}_\theta \end{bmatrix},$$

where it is again noted that $\mu_{U|Y=y_{\text{obs}}}$ is the conditional mean of the random effects U given $Y = y_{\text{obs}}$. Note that this conditional mean also depends on θ, although is not made explicit. We can perform a Cholesky decomposition on the above cross-product matrix, so that

$$\begin{bmatrix} \Lambda^T_\theta Z^T Z \Lambda_\theta + I & \Lambda^T_\theta Z^T X \\ X^T Z \Lambda_\theta & X^T X \end{bmatrix} = \begin{bmatrix} L_\theta & 0 \\ R^T_{ZX} & R^T_X \end{bmatrix} \begin{bmatrix} L^T_\theta & R_{ZX} \\ 0 & R_X \end{bmatrix}.$$

We may use this decomposition to rewrite the penalized weighted residual sum-of-squares as,

$$r^2(\theta, \beta, u) = r^2(\theta) + \|L^T_\theta (u - \mu_{U|Y=y_{\text{obs}}}) + R_{ZX}(\beta - \hat{\beta}_\theta)\|^2 + \|R_X(\beta - \hat{\beta}_\theta)\|^2.$$

The above derivation is an important expression in the underlying `lme4`. It relates the penalized residuals sum-of-squares with its minimum value and is extremely useful when we integrate out the random effects, which is required for maximum likelihood estimation. Note that we have simplified notation by writing $r^2(\theta, \hat{\beta}_\theta, \mu_{U|Y=y_{\text{obs}}})$ as $r^2(\theta)$.

Probability Densities

The residual sums-of-squares discussed in the previous section can be used to express various probability densities, which are required for maximum likelihood estimation,

$$\begin{aligned} f_{Y|U}(y_{\text{obs}}|u) &= \frac{1}{(2\pi\sigma^2)^{n/2}} \exp\left(-\frac{\rho^2(\theta,\beta,u)}{2\sigma^2}\right), \\ f_U(u) &= \frac{1}{(2\pi\sigma^2)^{q/2}} \exp\left(-\frac{\|u\|^2}{2\sigma^2}\right), \\ f_{Y,U}(y_{\text{obs}},u) &= \frac{1}{(2\pi\sigma^2)^{(n+q)/2}} \exp\left(-\frac{r^2(\theta,\beta,u)}{2\sigma^2}\right), \\ f_{U|Y}(u|y_{\text{obs}}) &= \frac{f_{Y,U}(y_{\text{obs}},u)}{f_Y(y_{\text{obs}})} \end{aligned}$$

where

$$f_Y(y_{\text{obs}}) = \int f_{Y,U}(y_{\text{obs}},u)du.$$

The log-likelihood to be maximized can therefore be written as,

$$\mathcal{L}(\theta,\beta,\sigma^2|y_{\text{obs}}) = \log f_Y(y_{\text{obs}}).$$

The integral representation of $f_Y(y_{\text{obs}})$ is explicitly written as,

$$\begin{aligned} f_Y(y_{\text{obs}}) = \frac{1}{(2\pi\sigma^2)^{(n+q)/2}} \exp\left[\frac{-r^2(\theta) - \|R_X(\beta-\hat{\beta}_\theta)\|^2}{2\sigma^2}\right] \\ \times \int \exp\left[-\frac{\|L_\theta^T(u-\mu_{U|Y=y_{\text{obs}}}) + R_{ZX}(\beta-\hat{\beta}_\theta)\|^2}{2\sigma^2}\right] du. \end{aligned}$$

The term inside the norm of the exponent being integrated is affine in u. Thus, a simple change of variables can ease the computation of the above integral,

$$v = L_\theta^T(u-\mu_{U|Y=y_{\text{obs}}}) + R_{ZX}(\beta-\hat{\beta}_\theta),$$

where the Jacobian of this affine transformation from u to v is $|L_\theta|$. Therefore we simply the integration problem as,

$$\frac{1}{(2\pi\sigma^2)^{(n+q)/2}} \exp\left[-\frac{r^2(\theta)+\|R_X(\beta-\hat{\beta}_\theta)\|^2}{2\sigma^2}\right] \int \exp\left[-\frac{\|v\|^2}{2\sigma^2}\right] |L_\theta|^{-1} dv,$$

which by properties of exponential integrands becomes,

$$\exp\mathcal{L}(\theta,\beta,\sigma^2|y_{\text{obs}}) = f_Y(y_{\text{obs}}) = \frac{|L_\theta|^{-1}}{(2\pi\sigma^2)^{n/2}} \exp\left[-\frac{r^2(\theta)+\|R_X(\beta-\hat{\beta}_\theta)\|^2}{2\sigma^2}\right].$$

Evaluating and Profiling the Deviance

We are able to explicitly profile β and σ out of the log-likelihood, to find a compact expression for the profiled deviance (negative twice the profiled log-likelihood) and the profiled REML criterion as a function of the relative covariance parameters, θ, only. Furthermore these criteria can be evaluated quickly and accurately. To estimate the parameters, θ, β, and σ^2, we minimize negative twice the log-likelihood, which can be written as,

$$-2\mathcal{L}(\theta,\beta,\sigma^2|y_{\text{obs}}) = \log|L_\theta|^2 + n\log(2\pi\sigma^2) + \frac{r^2(\theta)}{\sigma^2} + \frac{\|R_X(\beta-\hat{\beta}_\theta)\|^2}{\sigma^2}.$$

It is very easy to profile out β, it only enters the ML criterion through the final term, which is zero when $\beta = \hat{\beta}_\theta$ where $\hat{\beta}_\theta$ is found by solving the previous penalized least-squares problem. Therefore, we can write a partially profiled ML criterion as,

$$-2\mathcal{L}(\theta, \sigma^2|y_{\text{obs}}) = \log|L_\theta|^2 + n\log(2\pi\sigma^2) + \frac{r^2(\theta)}{\sigma^2}.$$

This criterion is only partially profiled because it still depends on σ^2. Differentiating this criterion with respect to σ^2 and setting the result equal to zero yields,

$$0 = \frac{n}{\hat{\sigma}^2_\theta} - \frac{r^2(\theta)}{\hat{\sigma}^4_\theta},$$

which leads to a maximum profiled likelihood estimate,

$$\hat{\sigma}^2_\theta = \frac{r^2(\theta)}{n}.$$

This estimate can be substituted into the partially profiled criterion to yield the fully profiled ML criterion,

$$-2\mathcal{L}(\theta|y_{\text{obs}}) = \log|L_\theta|^2 + n\left[1 + \log\left(\frac{2\pi r^2(\theta)}{n}\right)\right].$$

This expression for the profiled deviance depends only on θ. Although q, the number of columns in Z and the size of Σ_θ, can be very large indeed, the dimension of θ is small, frequently less than 10. The `lme4` package uses generic nonlinear optimizers to optimize this expression over θ to find its maximum likelihood estimate.

The REML Criterion

The REML criterion can be obtained by integrating the marginal density for Y with respect to the fixed effects,

$$\int f_Y(y_{\text{obs}})d\beta = \frac{|L_\theta|^{-1}}{(2\pi\sigma^2)^{n/2}}\exp\left[-\frac{r^2(\theta)}{2\sigma^2}\right]\int\exp\left[-\frac{\|R_X(\beta-\hat{\beta}_\theta)\|^2}{2\sigma^2}\right]d\beta,$$

which can be evaluated with the change of variables $v = R_X(\beta - \hat{\beta}_\theta)$, where the Jacobian determinant of the transformation from β to v is $|R_X|$. Therefore, we are able to write the integral as,

$$\begin{aligned}\int f_Y(y_{\text{obs}})d\beta &= \frac{|L_\theta|^{-1}}{(2\pi\sigma^2)^{n/2}}\exp\left[-\frac{r^2(\theta)}{2\sigma^2}\right]\int\exp\left[-\frac{\|v\|^2}{2\sigma^2}\right]|R_X|^{-1}d\beta \\ &= \frac{|L_\theta|^{-1}|R_X|^{-1}}{(2\pi\sigma^2)^{(n-p)/2}}\exp\left[-\frac{r^2(\theta)}{2\sigma^2}\right].\end{aligned}$$

A similar profile likelihood argument as the previous Section leads to the profiled REML criterion,

$$-2\mathcal{L}(\theta|y_{\text{obs}}) = \log\left(|L_\theta|^2|R_X|^2\right) + (n-p)\left[1 + \log\left(\frac{2\pi r^2(\theta)}{n-p}\right)\right].$$

Acknowledgments

We borrow materials from Faraway [2016], Pinheiro and Bates [2006], Agresti [2013], and the vignette for the `lem4` package

References

Alan Agresti. *Categorical Data Analysis*. John Wiley & Sons, 2013.

Julian J Faraway. *Extending the linear model with R: generalized linear, mixed effects and nonparametric regression models*. CRC press, 2016.

José Pinheiro and Douglas Bates. *Mixed-effects models in S and S-PLUS*. Springer Science & Business Media, 2006.